

UNIVERSIDAD DE CASTILLA-LA MANCHA
ESCUELA SUPERIOR DE INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Ingeniería del Software

TRABAJO FIN DE GRADO

Cronos: Creación de un sistema de procesamiento, almacenamiento, presentación y análisis de datos históricos

Jaime León Rosado

Julio, 2022



UNIVERSIDAD DE CASTILLA-LA MANCHA
ESCUELA SUPERIOR DE INFORMÁTICA

Departamento de Tecnologías y Sistemas de Información
Ingeniería del Software

TRABAJO FIN DE GRADO

Cronos: Creación de un sistema de procesamiento, almacenamiento, presentación y análisis de datos históricos

Autor: Jaime León Rosado

Tutor: José Antonio Cruz Lemus

Co-tutora: Ana Isabel Gómez Carretero

Julio, 2022

Plantilla guía de TFG
© Jaime León Rosado, 2022

Este documento se distribuye con licencia CC BY-NC-SA 4.0. El texto completo de la licencia puede obtenerse en https://creativecommons.org/licenses/by-nc-sa/4.0/.

La copia y distribución de esta obra está permitida en todo el mundo, sin regalías y por cualquier medio, siempre que esta nota sea preservada. Se concede permiso para copiar y distribuir traducciones de este libro desde el español original a otro idioma, siempre que la traducción sea aprobada por el autor del libro y tanto el aviso de copyright como esta nota de permiso, sean preservados en todas las copias.

Este texto ha sido preparado con la plantilla LaTeX de TFG para la UCLM publicada por Jesús Salido en GitHub[1] y Overleaf [2] como parte del curso *«LaTeX esencial para preparación de TFG, Tesis y otros documentos académicos»* impartido en la Escuela Superior de Informática de la Universidad de Castilla-La Mancha.



[1] https://github.com/JesusSalido/TFG_ESI_UCLM, DOI: 10.5281/zenodo.4574562

[2] https://www.overleaf.com/latex/templates/plantilla-de-tfg-escuela-superior-de-informatica-uclm/phjgscmfqtsw

Tribunal:

Presidente: _____

Vocal: _____

Secretario(a): _____

Fecha de defensa: _____

Calificación: _____

Presidente	Vocal	Secretario(a)
Fdo.:	Fdo.:	Fdo.:

A mi familia y amigos,
por acompañarme durante este trayecto

Resumen

El aumento de la importancia que se le da a la analítica de datos ha tomado relevancia conforme las organizaciones han ido comprobando las ventajas que esto trae para el negocio, y conforme a la mejora en las técnicas y procesos para la obtención, tratamiento, almacenamiento y visualización de los propios datos.

GlobalSuite, la empresa donde se ha realizado este proyecto a través de un convenio FORTE, detectó la necesidad de obtener valor de la inmensa cantidad de datos que maneja. Es por esto que, a partir de un proyecto interno previamente desarrollado que mostraba datos sobre el estado actual de la empresa, se pensó en que sería de utilidad aportar una dimensión temporal a estos datos para poder mostrar así posibles tendencias.

Para abordar este problema se ha creado una base de datos en MongoDB en la que se han almacenado datos extraídos periódicamente de dos fuentes distintas gracias a una API hecha en PHP, una de ellas una base de datos gestionada en MariaDB y la otra una base de datos en MongoDB. La API se ha utilizado para extraer los datos ejecutando consultas a las fuentes periódicamente y para procesar los resultados de estas consultas limpiándolos y agrupándolos antes de meterlos en la base de datos de destino. Una vez los datos estaban en el cluster de MongoDB creado para este proyecto, se ha utilizado la herramienta Mongo Charts para crear las gráficas, y estas se han añadido a una aplicación web desarrollada en React.

Como resultado se han añadido un total de catorce colecciones con datos y treinta y siete gráficas distintas para mostrar la evolución temporal de distintos valores explicados durante el documento. Para ello, se ha trabajado siguiendo principios de código limpio, calidad de software, gestión de proyectos y demás conocimientos adquiridos durante el Grado en Ingeniería Informática.

Abstract

The growing importance given to data analytics has become significant as organisations have become aware of the advantages it brings to the business, as well as the improvement in the techniques and processes for obtaining, processing, storing and visualising the data itself.

GlobalSuite, the company where this project has been carried out through a FORTE agreement, detected the need to obtain value from the immense amount of data it handles. This is why, based on a previously developed internal project that showed data on the current state of the company, it was considered that it would be useful to provide a time dimension to these data in order to show possible trends.

To address this problem, a database has been created in MongoDB in which data is periodically collected from two different sources thanks to an API made in PHP, one of them being a database managed in MariaDB, and the other a database in MongoDB. The API was used to extract the data by executing queries to the sources periodically and to process the results of these queries by cleaning and clustering them before putting them into the target database. Once the data was in the MongoDB cluster created for this project, the Mongo Charts tool was used to create the charts, and these were added to a web application developed in React.

As a result, a total of fourteen collections with data and thirty-seven different graphs have been added to show the temporal evolution of different values explained during the document. To do this, we have worked following principles of clean code, software quality, project management and other knowledge acquired during the Degree in Computer Engineering.

Agradecimientos

Tengo muchas cosas que agradecer a mucha gente.

A mi familia, el haber sido un gran apoyo durante todos estos años, no solamente los universitarios, sino en estos 23 años en los que hemos compartido tanto. No dudo que los que nos quedan por compartir sean muchos y tan buenos.

A los amigos del piso de David, por la confianza y el cariño demostrado. Se echan de menos los días de "Hoy como en tu casa". Se echan de menos los planes improvisados que siempre terminaban con alguna historia.

A los amigos del pueblo, por ser un punto de anclaje donde volver pese a la distancia. Aunque estemos tiempo sin vernos, siempre se conservan las ganas.

A toda la gente del Computeam, jugadores y aficionados, por los partidos que hacían más corta la semana. Miércoles de partido, y el resto de la semana de tercer tiempo.

También me gustaría agradecerle a José Antonio sus consejos y su apoyo durante este TFG, y a los compañeros de GlobalSuite, en especial a Ana y Lucía, su gran acogida en la empresa, su simpatía y su profesionalidad.

Me llevo de Ciudad Real una buena experiencia y gente muy valiosa.

Jaime León Rosado
Ciudad Real, 2022

Acrónimos

ANSI	:	American National Standards Institute
API	:	Application Programming Interface
AWS	:	Amazon Web Sevices
BI	:	Business intelligence
CI	:	Continous Integration
DBA	:	Database Administration
DCL	:	Data Control Language
DML	:	Data Manipulation Language
ELT	:	Extract Load Transform
ESI	:	Escuela Superior de Informática
ETL	:	Extract Transform Load
FORTE	:	FORTalecimiento y mejora de las competencias de empleabilidad de los graduados en Informática de la Escuela Superior de Informática de Ciudad Real
GRC	:	Gobierno, Riesgo y Cumplimiento
HTTP	:	Hypertext Transfer Protocol
IDE	:	Integrated Development Environment
ISO	:	International Organization for Standardization
JSON	:	JavaScript Object Notation
MQL	:	Mongo Query Language
NoSQL	:	Not Only SQL
OLAP	:	Online Analytical Processing
OLTP	:	Online Transactional Processing
ORM	:	Object–relational mapping
PCI DSS	:	Payment Card Industry Data Security Standard
PHP	:	PHP: Hypertext Preprocessor
RGPD	:	Reglamento General de Protección de Datos
SDK	:	Software Development Kit
SEQUEL	:	Structured English Query Language
SQL	:	Structured Query Language
SSL	:	Secure Sockets Layer
TCL	:	Transaction Control Language
TFG	:	Trabajo de Fin de Grado
TTL	:	Time To Live
UNE	:	Una Norma Española
UTC	:	Universal Time Coordinated
VDI	:	Virtual Development Infrastructure

Índice general

Resumen v

Abstract vii

Agradecimientos ix

Acrónimos xi

Índice de figuras xv

Índice de tablas xvii

Índice de listados xix

1. Introducción 1
- 1.1. Motivación . . . 1
- 1.2. GlobalSuite . . . 1
- 1.3. Forte . . . 2
- 1.4. Antecedentes: SIMBAD . . . 3
- 1.5. Estructura del documento . . . 4

2. Objetivo 5
- 2.1. Objetivo General . . . 5
- 2.2. Objetivos Específicos . . . 5
- 2.3. Objetivos Académicos . . . 5

3. Estado del arte 7
- 3.1. Bases de datos . . . 7
- 3.2. Lagos y Almacenes de Datos . . . 11
- 3.3. Cuadros de mandos . . . 12

4. Metodología 15
- 4.1. Scrum . . . 15
- 4.2. Flujo de Trabajo . . . 17
- 4.3. Marco tecnológico . . . 18

5. Resultados 23
- 5.1. Sprint 0 . . . 23
- 5.2. Sprint 1 . . . 24

5.3. Sprint 2 27
5.4. Sprint 3 30
5.5. Sprint 4 33
5.6. Sprint 5 34
5.7. Sprint 6 36

6. Conclusiones **49**
6.1. Consecución de objetivos parciales 49
6.2. Consecución de objetivos docentes 50
6.3. Consecución del Objetivo General 52
6.4. Adquisición de competencias específicas 52
6.5. El futuro de CRONOS 53
6.6. Valoración crítica 54

Bibliografía **55**

Índice de figuras

1.1. Vista de la aplicación SIMBAD 4

3.1. SQL Statements 9
3.2. *Top 10 Databases by Popularity* 10
3.3. NoSQL database types 11
3.4. ETL and ELT 12
3.5. Cuadrante de mágico de Gartner de plataformas de análisis e inteligencia de negocio 13
3.6. Ejemplo de cuadro en Mongo Charts 14

4.1. Metodología Scrum, obtenida en *https://agilehunter.com/que-es-scrum/* . . . 17
4.2. Flujo de Trabajo 18
4.3. Jerarquía MongoDB 19
4.4. Arquitectura de Jenkins 20
4.5. Jerarquía de Docker 21
4.6. Secuencia de acciones de Redux 22

5.1. Arquitectura de la aplicación 24
5.2. *Bucketing* 27
5.3. Gráficas asociadas a la tarea 5.2 30
5.4. Pestaña resultado de la tarea 5.4 32
5.5. Pestaña resultado de la tarea 5.3 32
5.6. Pestaña resultado de la tarea 5.5 32
5.7. Pestaña resultado de la tarea 5.8 34
5.8. Pestaña resultado de la tarea 5.9 34
5.9. Captura de SonarQube de la tarea 5.8 37
5.10. Ejemplo de descripción emergente 39
5.11. Pestaña resultado de la tarea 5.10 42
5.12. Pestaña resultado de la tarea 5.11 43
5.13. Pestaña resultado de la tarea 5.12 43
5.14. Pestaña resultado de la tarea 5.13 43
5.15. Primera parte de la pestaña resultado de la tarea 5.14 44
5.16. Segunda parte de la pestaña resultado de la tarea 5.14 44
5.17. Segunda parte de la pestaña resultado de la tarea 5.15 45
5.18. Segunda parte de la pestaña resultado de la tarea 5.16 45
5.19. Tablero la reunión retrospectiva correspondiente al *Sprint* 6. Algunos datos están censurados en base a los acuerdos con la empresa. 47

Índice de tablas

1.1. Proyectos FORTE realizados anteriormente 3

5.1. Sprint 1 - Actividad BBDD - Tamaño 25
5.2. Sprint 2 - Uso GS - Empresas - Datos Empresas 27
5.3. Sprint 3 - Uso GS-Empresas-Usuarios 30
5.4. Sprint 3 - Uso GS-Empresas-Licencias 31
5.5. Sprint 3 - Uso GS-Empresas-Licencias y empresas 31
5.6. Sprint 4 - Actividad BBDD-Tablas y esquemas-Tamaño 33
5.7. Sprint 4 - Uso GS - Encuestas - encuestas por tipo 33
5.8. Sprint 5 - Actividad BBDD - Tablas y esquemas - Leídas y modificadas . . . 35
5.9. Sprint 5 - Uso GS - Gestor Documental - tamaño gestor 35
5.10. Sprint 6 - Uso GS - Elementos Generales - activos, servicios y procesos . . 37
5.11. Sprint 6 - Uso GS - Elementos Generales - Scorecard y auditorías 38
5.12. Sprint 6 - Uso GS - Análisis de Riesgos - Total Análisis y Riesgos 38
5.13. Sprint 6 - Uso GS - Tickets - Tickets por Empresa 38
5.14. Sprint 6 - Uso GS - Planes - Planes por Tipos 39
5.15. Sprint 6 - Uso GS - Planes - Activación de la Crisis 39
5.16. Sprint 6 - Uso GS - Empresas - Datos actuales de empresas 40
5.17. Sprint 6 - Modificar tabla Surveys 41
5.18. Sprint 6 - Eliminar el *endpoint* de ayuda al desarrollo 41
5.19. Relación entre las tareas del Sprint 6 y las capturas de pantalla resultantes de cada una de ellas 42

Índice de listados

5.1. Ejemplo de creación de coleccion *Time Series* 26
5.2. Ejemplo de documento con Bucketing 28
5.3. Ejemplo uso de array_merge_recursive 29
5.4. Ejemplo uso de array_map_no_complaciente 29
5.5. Ejemplo uso de array_map_complaciente 29
5.6. Antigua estructura del comando 31
5.7. Nueva estructura del comando 31
5.8. Query para sacar los datos de la última fecha 39

CAPÍTULO 1

Introducción

1.1. MOTIVACIÓN

GlobalSuite es una empresa que genera un gran volumen de datos de valor, desde datos sobre las empresas a las que vende sus licencias, hasta los propios datos que se generan durante el desarrollo del software que vende la compañía. Todos estos datos son almacenados y gestionados por un equipo destinado únicamente a la administración de bases de datos.

Esos datos son muy valiosos, pero el valor que aportan a la compañía es nulo si no se crean herramientas para extraerlo y visualizarlo. Es por esto que se comenzó una linea de trabajo dentro de la organización para crear una aplicación web donde ver información relevante para el negocio. Esto permite controlar el estado de la aplicación en tiempo real, pero la información es volátil y no existe ninguna forma de visualizar datos antiguos, es decir, se pierde una dimensión tan relevante como es la del tiempo, y con ella la capacidad de detectar tendencias, anomalías, y tomar decisiones basadas en este conocimiento temporal.

Es por esto que nace el proyecto *CRONOS*, para aportar esta dimensión temporal a la información recogida y crear una fuente de datos históricos que se pueden consultar y tratar de distintas formas para aportar aun más valor a los datos generados por GlobalSuite, que sirva, por ejemplo, a los desarrolladores para ver el estado de las bases de datos, a los directivos para ver el número de licencias de la compañía, a los clientes para ver qué módulos de la aplicación usan más, y un largo etcétera que puede aumentar conforme aumenten la familiarización de los interesados con la herramienta, y conforme avance el campo del análisis de datos.

1.2. GLOBALSUITE

GlobalSuite, anteriormente conocida como AUDISEC, es una empresa nacida en Manzanares (Ciudad Real) en el año 2007 fruto del TFG de un compañero de la ESI (Escuela Superior de Informática), que creó la primera versión del software que cimienta la actividad del negocio.

La compañía esta especializada en GRC (Gobierno, Riesgo y Cumplimiento), es decir, en el modelo que trata de ayudar a quien lo implanta a cumplir sus objetivos, reducir la

incertidumbre y actuar con integridad mediante la alineación de los objetivos de negocio con los riesgos derivados de la actividad y el cumplimiento de los requisitos legales y normativos.

GlobalSuite ha logrado un importante crecimiento gracias a la herramienta homónima a la compañía, que ofrece a sus más de 2000 clientes, de más de 35 países, una serie de herramientas que van desde la posibilidad de definir los procesos de la empresa, hasta mostrar métricas e indicadores usando datos de las propias organizaciones.

Esto aporta a los más de 20 000 usuarios la posibilidad de centralizar las siguientes características:

- **Seguridad**: Asistencia a la gestión de las certificaciones ISO-27001 (Seguridad de la información), ISO-27018(Privacidad en la nube), ISO-27701(Privacidad de la información personal) e ISO-28000(Seguridad de la cadena de suministro), así como adecuar la organización al esquema nacional de seguridad y PCI DSS (Payment Card Industry Data Security Standard).
- **Gestión de Riesgos**: Asistencia a la gestión de la certificación ISO-31000(Riesgos empresariales) así como soluciones personalizadas acerca de ciberseguridad y ciberriesgo
- **Continuidad de Negocio**: Asistencia a la gestión de la certificación ISO-22301(Plan de continuidad de negocio) e ISO-27031(Continuidad tecnológica)
- **Gestión del cumplimiento**: Asistencia a la gestión de la certificación ISO-37301(Compliance Management), ISO-37301 (Compliance), ISO-19600 (Compliance), ISO-37001 (Antisoborno), UNE-19601 (Compliance Penal), así como la existencia del Canal de denuncias, donde se pueden gestionar denuncias de forma anónima,
- **Privacidad y Protección de datos**: Ayuda a las empresas a implantar y adaptarse a la RGPD(Reglamento General de Protección de Datos), así como a estar listas para las auditorías.
- **Gestión de Auditorías**: Ayuda a la realización de auditorías centralizando evidencias, apoyando la trazabilidad de los trabajos de auditorías, y conteniendo un flujo de trabajo para su optimización.

Esto, y la continua mejora que experimenta la aplicación, son una parte clave para el crecimiento que ha tenido esta empresa.

1.3. FORTE

FORTE (FORTalecimiento y mejora de las competencias de empleabilidad de los graduados en Informática de la Escuela Superior de Informática de Ciudad Real) es el nombre que se le da al convenio entre la Escuela Superior de Informática y algunas empresas, cuyo objetivo es la integración del estudiante en un proyecto real sobre el que pueda desarrollar su TFG. Al estudiante que realiza el FORTE se le asigna un tutor académico, típicamente un miembro de la plantilla de profesores de la ESI, y un tutor por parte de la empresa, siendo el tutor del

TFG encargado de todo lo relacionado al desarrollo correcto del TFG, y el tutor de prácticas encargado de la integración dentro de la empresa y los aspectos técnicos del proyecto.

Para realizar el FORTE, el requisito principal del alumno es que únicamente tenga el TFG, las prácticas y una o dos asignaturas como máximo por cursar, además de finalizar exitosamente el proceso de selección, que consiste primero en la comunicación al encargado por parte de la ESI de que se quieren realizar las prácticas, luego las empresas interesadas en adquirir alumnos lo comunicarán al responsable. Este responsable envía a los alumnos los proyectos, y los alumnos envían una lista de los proyectos a los que les gustaría acceder. Una vez la empresas reciben la lista de los alumnos interesados en el proyecto, estas se ponen en contacto con los alumnos para realizar una serie de entrevistas para ponderar a los candidatos. Una vez han realizado la priorización, la envían al responsable de la ESI, que se encarga de comunicarlo a los alumnos y realizar la asignación final.

Como se puede ver en la tabla 1.1, GlobalSuite ya ha realizado anteriormente dos FORTEs. Estos proyectos fueron un éxito y han generado productos útiles para la organización que, entre otras cosas, han dado posibilidad a que se realice este proyecto. La empresa integra a los alumnos en el modelo interno de trabajo, centrándose en que este pueda entregar su TFG a tiempo y con una calidad profesional, lo que hace que la experiencia sea buena y de provecho.

Nombre	Título del TFG	Fecha
Lucía Alfonso García	Sistema de análisis de actividad de la base de datos de GlobalSUITE	julio 2021
Sara Sevillano Fernández	Sistema de Validación funcional en aplicaciones web (GlobalSUITE)	septiembre 2021

Tabla 1.1: Proyectos FORTE realizados anteriormente

1.4. ANTECEDENTES: SIMBAD

La necesidad de controlar los datos generados por GlobalSuite no es algo nuevo, y es por ello que en junio de 2021 ya se realizó un FORTE para la creación de una aplicación web que contuviese un conjunto de métricas relativas al estado de la plataforma de GlobalSuite en el momento de la medición. Los resultados obtenidos de la creación de SIMBAD, plantearon la posibilidad de que en un futuro se añadiese la dimensión temporal a estos datos para ver la evolución e incluso poder detectar tendencias.

El resultado fue positivo, y el proyecto dio como resultado una aplicación web funcional (ver Figura 1.1) en la que se mostraban gráficas interactivas con información de interés como el tamaño de las bases de datos, el número de licencias de la aplicación que había activas o el tiempo medio de duración de las consultas a la base de datos, entre otras.

El proyecto CRONOS, objetivo del presente TFG, nace como continuación de SIMBAD, añadiéndole la dimensión temporal para posibilitar la consulta de datos históricos de ciertas



Figura 1.1: Vista de la aplicación SIMBAD

métricas que proporcionen información relevante sobre las tendencias que siguen los datos recogidos por la empresa y así poder analizar la situación de esta.

1.5. ESTRUCTURA DEL DOCUMENTO

- **Capítulo 1: Introducción**: Presentación de la motivación del TFG y el contexto sobre el que se realiza.
- **Capítulo 2: Objetivo**: En esta sección se presenta tanto del objetivo general como de los objetivos específicos y académicos que se deben completar para alcanzarlo.
- **Capítulo 3: Estado del Arte**: Explicación de las herramientas y tecnologías necesarias para entender el TFG así como las que han sido utilizadas para el correcto desarrollo del proyecto.
- **Capítulo 4: Metodología**: Descripción de la metodología definida para la gestión y desarrollo del proyecto.
- **Capítulo 5: Resultados**: Presentación de los resultados obtenidos tras el desarrollo del proyecto.
- **Capítulo 6: Conclusiones**: Se presenta un análisis de las conclusiones obtenidas del proyecto, revisión de los objetivos justificando su grado de consecución, así como las líneas futuras de trabajo sobre este TFG y una valoración crítica de todo el proceso que ha rodeado al mismo.

Conviene resaltar que, si bien se incluye un apartado específico de bibliografía al final del documento, muchas de las referencias, tanto bibliográficas como de cualquier otro tipo de contenido que se han utilizado para la elaboración del presente documento, se adjuntan *ad hoc* como notas al pie de las distintas páginas, para intentar mejorar su legibilidad y utilizar de un modo más eficiente la navegación de los recursos online.

CAPÍTULO 2

Objetivo

La finalidad de este capítulo es definir el objetivo general de este TFG, así como los objetivos específicos y académicos que se deberán cumplir para alcanzarlo.

2.1. OBJETIVO GENERAL

El objetivo principal de este trabajo es crear un sistema de almacenamiento, procesamiento y presentación de datos históricos relacionados con el *software* GlobalSuite, que permita identificar e implementar mejoras técnicas y de negocio analizando tendencias sobre el uso y gestión del producto.

2.2. OBJETIVOS ESPECÍFICOS

Para conseguir el objetivo general de este TFG, se deberían de cumplir, al menos, los objetivos enmarcados en la enumeración inferior:

1. Creación de la base de datos que contenga los datos históricos.
2. Análisis de las fuentes de datos.
3. Implementación de métricas.
4. Creación de un cuadro de mando para la presentación de los datos.

2.3. OBJETIVOS ACADÉMICOS

Estos objetivos muestran la aplicación de los conocimientos adquiridos durante las diferentes asignaturas del grado que se deberían aplicar durante el desarrollo del proyecto.

1. Capacidad de trabajo con bases de datos tanto relacionales como no relacionales. Creación de estas, realización de consultas y búsqueda de registros.
2. Seguimiento de metodologías ágiles con su respectiva adaptación al contexto del equipo.
3. Seguimiento de las buenas prácticas a la hora de programar, manteniendo un código limpio, legible, comprensible, documentado y de calidad.

4. Aplicación de los conocimientos adquiridos sobre la creación, necesidad y utilidad de los cuadros de mando.
5. Desarrollo de sistemas que puedan procesar y responder peticiones HTTP.
6. Uso de software de control de versiones.

CAPÍTULO 3

Estado del arte

En este apartado se describirán distintas tecnologías que se han utilizado durante el trabajo o que se han tenido en cuenta para su desarrollo. Primero se describen las bases de datos y se profundiza tanto en las bases de datos relacionales como en las no relacionales, así como en los conceptos de almacén de datos *Data Warehouse* y lago de datos *Data Lake*. Por último, se comentan los principales aspectos relativos a los cuadros de mando.

3.1. BASES DE DATOS

El continuo desarrollo de la informática así como las mejoras tanto en los sistemas de información como en la infraestructura que los soporta, ha creado la necesidad de almacenar y organizar la información que se genera y/o se necesita. El estudio de las bases de datos es una disciplina en la que aun hoy se sigue avanzando tanto teórica como prácticamente, a pesar de que ha sido un reto para el que se han planteado distintas soluciones durante más de 70 años.

Al principio, las primeras implementaciones fueron rudimentarias, aunque sentaron las bases para las tecnologías modernas. Estas soluciones consistían principalmente en el uso de en ficheros, en los que la información se almacenaba en formato de texto sobre el que se realizan recorridos posteriores para encontrar los registros deseados. Esta solución ofrecía como principal ventaja que añadía persistencia al *software*, algo absolutamente novedoso y necesario en la época, pero no tiene en cuenta factores que a día de hoy son esenciales como la eficiencia en las búsquedas, la relación entre los datos o las operaciones con los datos incluidos en estos ficheros.

Con el paso de los años, Edgar Frank Codd advirtió en su artículo *'A Relational Model of Data for Large Shared Data Banks'* [2] sobre algunos de los problemas que suponía depender, para la gestión y el manejo de los datos, de un sistema tan arcaico como el que existía por ese entonces, quedando descritas algunas ideas que se resolverían con el modelo relacional que venía a presentar, como que la representación interna de los datos y las actividades que realizaban los usuarios de los sistemas debían ser independientes entre sí, o las dependencias de ordenación, indexado, y enrutamiento de acceso [2]. El artículo continúa describiendo las

bases del modelo relacional, esto es, el concepto de relación y el cómo deben ser entendidos los datos que se almacenen bajo este modelo, así mismo, también aporta el concepto de normalización de una base de datos seguido de un ejemplo. Por último, el autor explica cómo obtener tanto redundancia como consistencia en una base de datos. Dos conceptos que a día de hoy siguen siendo muy importantes. El primero sobre todo en bases de datos críticas (como puede ser la de un banco, como dice el título del artículo) y el segunda por la implicación que tiene, que todos los datos que guardados sean válidos, mantenga la integridad de estos, y si no es posible, se evite su almacenamiento.

Esto último nos lleva a los principios ACID (de sus siglas en inglés descritas en la enumeración inferior), que describen las características que tiene que tener un sistema de gestión de bases de datos relacional para considerarse como tal. El incumplimiento de estos principios podría generar errores en la validez de los datos. En concreto las cuatro letras corresponden a:

- **Atomicidad (*Atomicity*)**
  Indica que una transacción es una unidad atómica de trabajo que, o bien se ejecuta toda de una vez, o si no, no se ejecuta.
- **Consistencia (*Consistency*)**
  Antes y después de cada transacción, la base de datos debe ser consistente, es decir, se debe mantener la integridad de los datos.
- **Aislamiento (*Isolation*)**
  Las transacciones ocurren independientemente sin interferencias entre ellas.
- **Durabilidad (*Durability*)**
  Los cambios que se ejecuten mediante las transacciones deben ser permanentes y deben permaneces almacenados aunque ocurra algún fallo.

Codd adelanta también la necesidad de un nuevo lenguaje para realizar consultas en bases de datos cuyo modelo fuera el relacional, y es así donde empresas como IBM u Oracle empiezan a investigar al respecto, llegando gracias a la primera el lenguaje de consultas SEQUEL(*Structured English Query Language*) en el año 1977, y entrando al mercado de la mano de la segunda en el año 1979. No fue hasta 1986 cuando ANSI (*American National Standards Institute*) normalizó este lenguaje bajo el nombre de SQL (*Structured Query Language*). Con el paso del tiempo, SQL ha sufrido numerosas revisiones siendo en el año 2016 la última.

SQL permite multitud de operaciones (ver Figura 3.1), todas ellas acompañadas de una serie de modificadores que amplían aún más si cabe la utilidad del lenguaje. Estas operaciones están agrupadas en cuatro:

- **DDL (*Data Definition Language* o Lenguaje de Definición de Datos)**
  Permite definir, editar y borrar las estructuras que contienen los datos, típicamente tablas.

- **DML (*Data Manipulation Language* o Lenguaje de Manipulación de Datos)**
  Permite consultar, insertar, modificar y borrar los datos.
- **DCL (*Data Control Language* o Lenguaje de Control de Datos)**
  Permite gestionar los roles de los datos, que permiten (o no) realizar operaciones de los distintos lenguajes descritos en esta enumeración.
- **TCL (*Transactional Control Language* o Lenguaje de Control de Transacciones)**
  Permite administrar las transacciones realizadas en la base de datos.

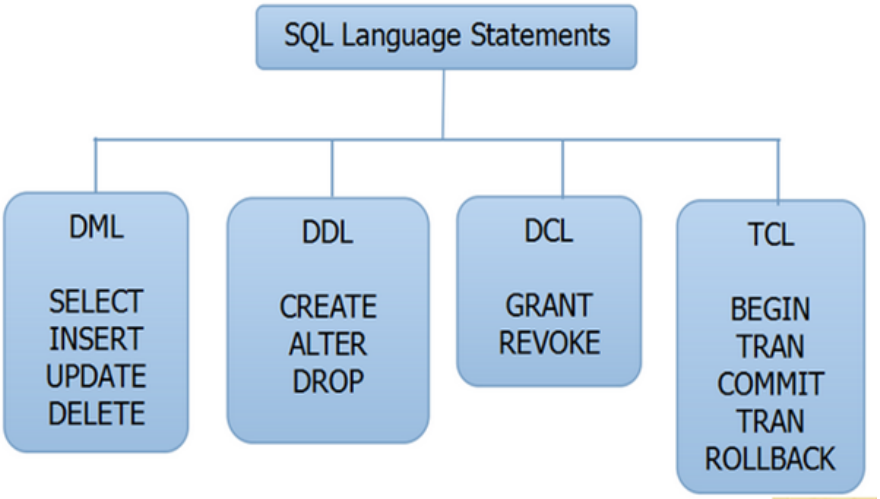


Figura 3.1: SQL Statements

Pese a la multitud de beneficios que presentaba esta solución, los requisitos modernos de algunos tipos de sistemas, como aquellos que trabajan con datos en tiempo real, o aquellos en los que la información se estructura mejor en otro tipo de modelos, quedaban insatisfechos. Es por esto que aparecieron otro tipo de tecnologías que no dependían del modelo relacional, y aportaban otras ventajas e inconvenientes, pero que, al ayudar a resolver otros problemas, tenían cabida en el mundo comercial. A las tecnologías que no utilizaban el modelo relacional y SQL como su principal característica, se las llamó tecnologías NoSQL (*Not Only SQL*), y se describirá más en profundidad en el apartado 3.1.1.

Actualmente, de acuerdo a db-engines[1], podemos aseverar que vivimos en un entorno híbrido donde tanto las bases de datos relacionales, como las que usan tecnologías NoSQL coexisten, ya que, como muestran los datos de la Figura 3.2, en los 10 primeros puestos de bases de datos más populares encontramos 7 relacionales, una documental, una clave-valor y un motor de búsqueda. Esto son datos de popularidad y no de uso, pero con ello podemos ver que los desarrolladores han entendido que existen soluciones variadas a los distintos problemas del mundo real, y que no necesariamente tienen que ser resueltos con bases de datos relacionales.

Estos sistemas que permiten tanto la inserción rápida de datos, como las consultas

[1]https://db-engines.com/en/ranking

Rank Mar 2022	Rank Feb 2022	Rank Mar 2021	DBMS	Database Model	Score Mar 2022	Score Feb 2022	Score Mar 2021
1.	1.	1.	Oracle	Relational, Multi-model	1251.32	-5.51	-70.42
2.	2.	2.	MySQL	Relational, Multi-model	1198.23	-16.45	-56.59
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	933.78	-15.27	-81.52
4.	4.	4.	PostgreSQL	Relational, Multi-model	616.93	+7.54	+67.64
5.	5.	5.	MongoDB	Document, Multi-model	485.66	-2.98	+23.27
6.	6.	↑ 7.	Redis	Key-value, Multi-model	176.76	+0.96	+22.61
7.	7.	↓ 6.	IBM Db2	Relational, Multi-model	162.15	-0.73	+6.14
8.	8.	8.	Elasticsearch	Search engine, Multi-model	159.95	-2.35	+7.61
9.	9.	↑ 10.	Microsoft Access	Relational	135.43	+4.17	+17.29
10.	10.	↓ 9.	SQLite	Relational	132.18	+3.81	+9.54

Figura 3.2: *Top 10 Databases by Popularity*

complejas en un tiempo razonable de tiempo, son perfectos para gestionar entornos de *Big Data*. *Big Data* es un término utilizado para denominar un conjunto masivo de datos que cumple ciertas características. Estas características han variado con el tiempo, pero las tres iniciales, que se mantienen, son:

- **Volumen.** Para que sea considerado *Big Data*, el volumen de los datos tiene que ser grande. Un tamaño mayor o mucho mayor que el de bases de datos tradicionales.
- **Velocidad.** El procesamiento de los datos debe ser rápido, sobre todo en su inserción, porque la creación de datos será algo continuo.
- **Variedad.** Los datos recogidos provenir de distintas fuentes, ya sean dispositivos o sistemas, y pueden tener distintos formatos, cadenas, numéricos, vídeos, etc.

Además, se han ido añadiendo características, como la veracidad, la viabilidad, la visualización o el valor de los datos, que también deben ser tenidas en cuenta al desarrollar soluciones con *Big Data*.

Por último, hay que destacar que al *Big Data* lo rodean técnicas y tecnologías pensadas para crear, almacenar y extraer los datos de una forma rápida y eficiente, que han que sea útil para muchas compañías plantear soluciones que cuenten con esto.

3.1.1. Bases de datos no relacionales

Las bases de datos no relacionales, o NoSQL se caracterizan por no depender del modelo relacional y funcionar sobre marcos teóricos distintos, donde los datos se organizan con otro tipo de estructuras más flexibles, pensados para solucionar problemas para los que las bases de datos relacionales no aportaban una solución correcta, como un muy alto tráfico de datos, escalabilidad horizontal, complejidad y coste de la creación de *clusters*, requisitos de la computación en la nube, como la estructura en la que se manejan los datos en las API (*Application Programming Interface*), y que, mientras el modelo relacional se pensó para un despliegues centralizados y no distribuidos, las tecnologías NoSQL nacen teniendo estas nuevas topologías [4].

Existen varios tipos de bases de datos no relacionales (ver Figura 3.3):

- **Documentales.** Utiliza el formato JSON para almacenar la información. Estas bases de datos destacan por ser flexibles y semi-estructuradas. Como hemos visto en la Figura 3.2, la quinta base de datos más popular, MongoDB, es de este tipo.
- **Clave-valor.** Estas bases de datos, como dice su propio nombre, almacenan la información en pares clave-valor, donde la clave es única y para cada una se asigna un valor. Este tipo de bases de datos son muy útiles para entornos en los que la velocidad de lectura y escritura deban ser muy grandes.
- **Columnares.** Tienen la peculiaridad de que la organización de los datos se organizan por columnas en vez de por filas. Esto otorga la capacidad de añadir velocidad a las transacciones y al análisis de los datos.
- **Orientadas a grafos.** Las bases de datos orientadas a grafos utilizan las teoría de grafos para almacenar la información en nodos, junto con las relaciones entre estos.

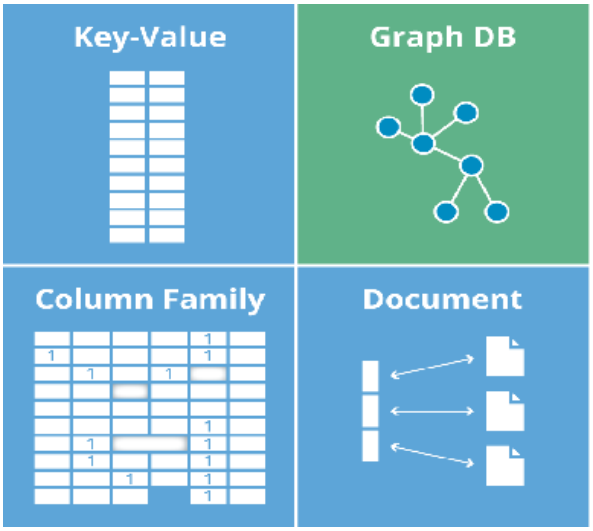


Figura 3.3: NoSQL database types

3.2. LAGOS Y ALMACENES DE DATOS

Un lago de datos es un repositorio donde se almacenan datos en su formato nativo, ya sean datos estructurados, semi-estructurados o no estructurados. Este tipo de almacenes se crea para manejar un gran volumen de datos que llegan a gran velocidad, y que más tarde serán procesados para obtener valor de ellos. Por su parte, un almacén de datos contiene datos altamente estructurados, preprocesados y que entran al almacén con menor periodicidad [3].

Mientras que las ventajas del Almacén de Datos residen en la velocidad en la que se pueden extraer los datos de él, y lo organizados que están los datos dentro de él, el Lago de Datos se beneficia de que la carga es más rápida y flexible debido a la falta de estructuración de las colecciones.

Al hilo de lo anterior, se pueden explicar los conceptos de ETL (*Extract, Transform, Load*) y ELT (*Extract, Load, Transform*) (ver Figura 3.4), que corresponden al orden como se tratan los datos en los Almacenes de Datos y los Lagos de Datos. ETL es el orden usado en los Almacenes de Datos, donde la transformación (etapa en la que los datos se organizan), se hace antes de la carga. Por otro lado, el orden ELT se utiliza para los Lagos de Datos, donde la transformación se hace después de la carga.

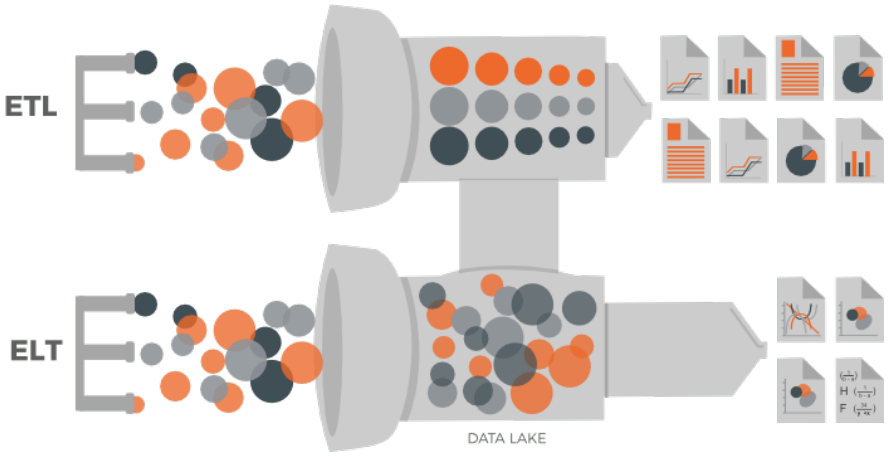


Figura 3.4: ETL and ELT

Alrededor de estas tecnologías se han construido sistemas, cada uno con sus requisitos y sus necesidades. En el mundo de las bases de datos en linea, son dos los tipos de sistema que han trascendido [1], estos son:

- **OLTP (*Online Transaction Processing*).** Se caracteriza por ser útil para el manejo de una gran cantidad de transacciones, hechas por una gran cantidad de usuarios, en un tiempo asequible. Este tipo de transacciones son comúnmente de creación, eliminación o borrado de datos, y los usuarios de internet las utilizan diariamente en acciones como las compras *online* o los cambios de contraseñas.
- **OLAP (*Online analytical processing*).** Permite al usuario ver el resultado de consultas complejas, que suelen realizarse menos a menudo, y que suelen ser multidimensionales. Para adquirir velocidad en estas consultas, se suelen utilizar los cubos, que son conjuntos de consultas precalculadas en diferentes dimensiones para tener preparados los resultados por si son requeridos.

3.3. CUADROS DE MANDOS

Monitorizar y analizar ciertas métricas es de gran interés para las empresas, por ello, para mantener organizada esta información, se crean los cuadros de mando. Son de gran utilidad para ver qué esta pasando dentro de la compañía, y tomar decisiones basadas en el conocimiento que se puede extraer de ellos.

Además de las métricas, los cuadros de mando suelen también contener indicadores,

que aportan una evaluación de las métricas. Es decir, si una métrica muestra el tiempo que tarda una consulta en ejecutarse, el indicador diría si eso es mucho, poco, o está dentro de lo normal.

Los cuadros de mando son la cara visible de la BI (*Business Intelligence* o Inteligencia de Negocio), que es el conjunto de herramientas, técnicas y datos que se usan para transformar los datos en información y la información en conocimiento de los interesados. En este caso, los cuadros de mando nos ayudan con la visualización de estos datos, permitiendo tanto la visualización de las distintas gráficas como su filtrado según las dimensiones posibles.

Los cuadrantes mágicos de Gartner clasifican las empresas de un sector en dos ejes. Según la posición en la que se encuentre una empresa, puede ser aspirantes, líderes, empresas de nicho y visionarias, cada una con sus características. En el caso de la BI, un cuadrante mágico se muestra en la Figura 3.5.



Figura 3.5: Cuadrante de mágico de Gartner de plataformas de análisis e inteligencia de negocio

Además de estas, existen otras alternativas como Mongo Charts, integrado directamente dentro de MongoDB Atlas y que permite la creación y exportación de tanto los cuadros de mando como de las métricas individuales dentro de estos. Un ejemplo de cuadro de mandos usando Mongo Charts se muestra en la Figura 3.6.

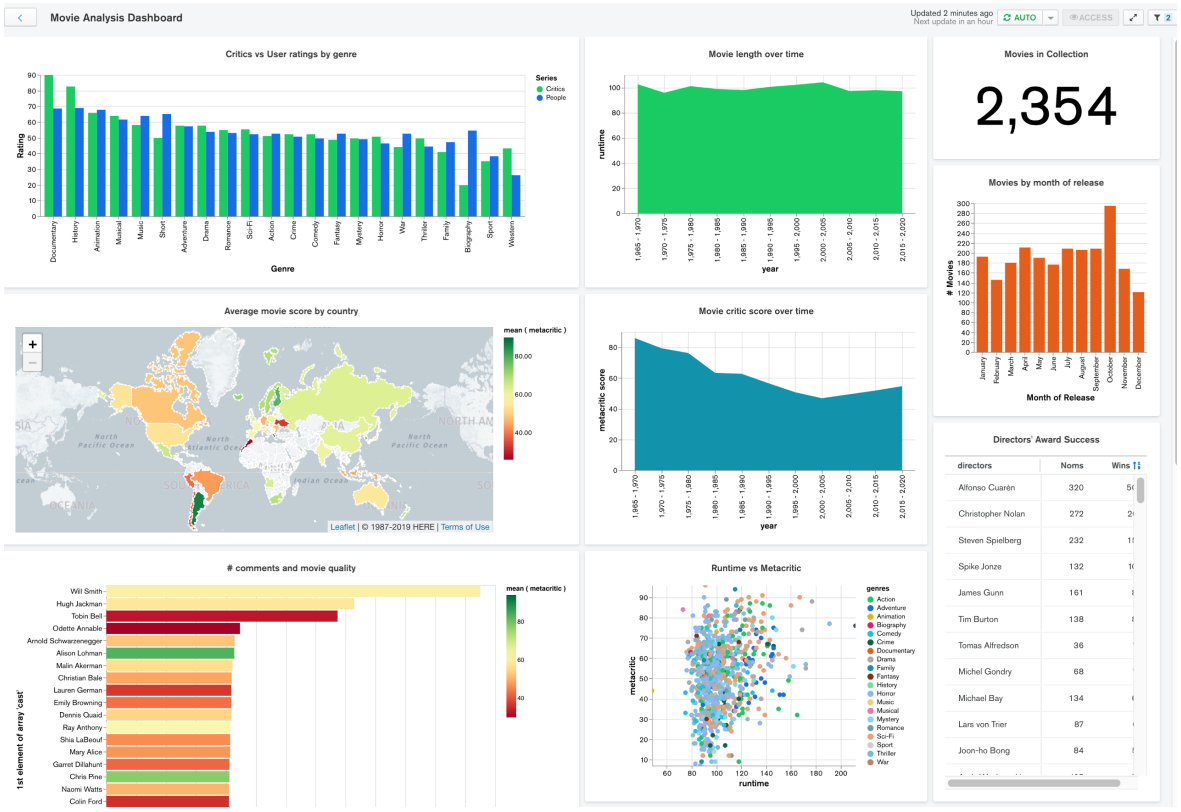


Figura 3.6: Ejemplo de cuadro en Mongo Charts

CAPÍTULO 4

Metodología

En este capítulo se detalla la metodología de gestión de proyectos empleada, el flujo de trabajo que se ha seguido y las tecnologías que se han utilizado en el proyecto.

4.1. SCRUM

Scrum es una metodología ágil gestión de proyectos centrada en maximizar el valor que obtenemos en proyectos complejos. Esta metodología no se basa en el seguimiento de un plan sino en la adaptación continua a las circunstancias de la evolución del proyecto, por lo que es interesante aplicarla en proyectos en los que sabemos que va a haber cambios.

La metodología divide el tiempo en sprints, que son franjas de tiempo de una a cuatro semanas, con una duración constante durante la que se ejecutan todas las reuniones de Scrum (ver apartado 4.1.4). En el contexto del proyecto actual, los Sprints son de dos semanas.

Scrum se cimienta sobre los principios de:

1. **Transparencia.** Mantiene a todos los responsables actualizados con los resultados del proyecto.
2. **Inspección.** Frecuentemente se revisa el progreso hacia el objetivo para detectar desviaciones.
3. **Adaptación.** En caso de detectar desviaciones, se ajustan.

Además, Scrum abraza los valores del compromiso, coraje, apertura, enfoque y respeto, que permiten que la metodología se ejecute de una forma adecuada y mantienen el correcto ambiente de trabajo.

Para entender el funcionamiento de Scrum se necesitan conocer los conceptos de ítem, pila, rol y reunión, ya que son parte clave de la metodología.

4.1.1. Ítem

Un ítem es un artefacto o elemento de trabajo que describe lo que hay que hacer para generar valor. Un ejemplo de ítem sería una historia de usuario, que contiene campos generales como

el título de esta o una descripción, así como campos más específicos como la estimación, el valor de la historia de usuario o la definición de hecho.

4.1.2. Pilas

Los ítems (ver seccción 4.1.1) que hay que completar se almacenan en dos pilas distintas, con características diferentes. Estas pilas son:

1. **Pila de producto.** Lista de requisitos de usuario que a partir de la visión inicial del producto crece y evoluciona durante el desarrollo.
2. **Pila de sprint.** Lista de trabajos que debe realizar el equipo durante el Sprint.

Durante el transcurso de este proyecto, las pilas han sido mantenidas en Jira.

4.1.3. Roles

Scrum trabaja en equipos auto-organizados, multifuncionales y colaborativos de 3 a 9 personas. Dentro de una organización puede haber varios equipos, y dentro de cada equipo destacan 3 roles, cada uno con una función distinta:

1. **Scrum Master**. Encarnado en este contexto por **Armando Ferrero Camargo**, el Scrum Master es el responsable del funcionamiento de la metodología, forma y entrena a los participantes, introduce Scrum en la cultura de la empresa, facilita los eventos o reuniones de Scrum (ver apartado 4.1.4) y elimina impedimentos entre sus funciones principales
2. **Product Owner**. Representado por **Ana Isabel Gómez Carretero** es el encargado de representar a los interesados en el producto y lograr el mayor valor para los clientes, usuarios, y en general todos los implicados. Una función muy importante, gestiona la pila del producto (ver apartado 4.1.2), donde crea ítems (ver apartado 4.1.1) priorizados, expresados de manera clara y entendidos por el equipo de desarrollo.
3. **Equipo de Desarrollo**. Compuesto por **Lucía Alfonso García** y **Jaime León Rosado**, es el encargado de generar el incremento.

4.1.4. Reuniones

Para el correcto funcionamiento de Scrum, la comunicación entre roles (ver sección 4.1.3) y aportar la agilidad necesaria para generar valor de una forma flexible, Scrum define una serie de reuniones en el flujo de trabajo.

1. **Planificación.** La reunión de planificación ocurre cada vez que se va a iniciar un sprint nuevo, en ella se selecciona qué elemento de la pila de producto (ver apartado 4.1.2) pasas a la pila del sprint, se define cual va a ser el objetivo del sprint, y se ve que aporta más valor.

2. **Diaria.** La reunión diaria ocurre diariamente, es de una duración inferior a quince minutos, y en ella se comenta lo que se hizo el día anterior, lo que se hará el día de la reunión, y los problema que han surgido.
3. **Revisión.** Esta reunión consiste en la presentación del incremento realizo por parte del equipo de desarrollo para que el *Product Owner* este al tanto de él.
4. **Retrospectiva.** La retrospectiva permite que el equipo se inspeccione a sí mismo y cree un plan de mejora para el siguiente sprint. En ella se identifican tanto las cosas que salieron bien como las que salieron mal, y se crea un plan de mejora de cara al siguiente sprint.

Las reuniones se han mantenido de forma telemática debido al régimen de teletrabajo.

Scrum es una metodología adaptable, que permite, dentro de unos límites, que cada entidad que la ejecute varíe unas características u otras para adaptarla a su realidad. Todo esto es mostrado en la Figura 4.1, en la que se muestra el flujo de ejecución de Scrum resumido de forma general. El *product owner* define las historias de usuario, las añade priorizadas a la pila de producto, ocurre la reunión de planificación donde se define la pila del sprint, y comienza el sprint. Durante el sprint se ejecutan las reuniones diarias y se genera el incremento. Una vez terminado el sprint, ocurren las reuniones de revisión y retrospectiva, y se vuelve a ejecutar la reunión de planificación para comenzar otra iteración del ciclo.

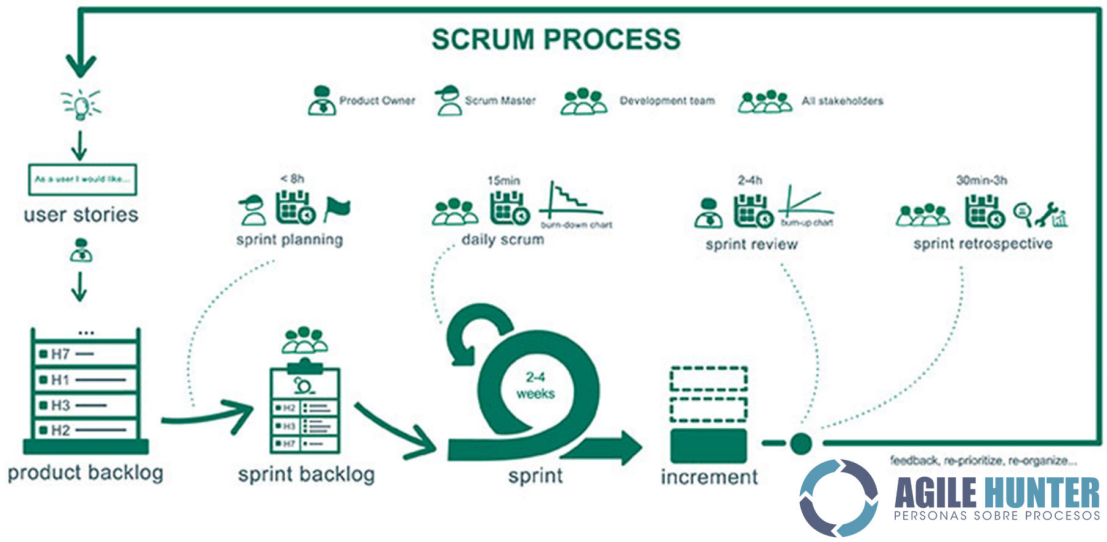


Figura 4.1: Metodología Scrum, obtenida en *https://agilehunter.com/que-es-scrum/*

4.2. FLUJO DE TRABAJO

Durante el desarrollo se han establecido tareas cuyo ciclo de vida (ver Figura 4.2) se explican en esta sección.

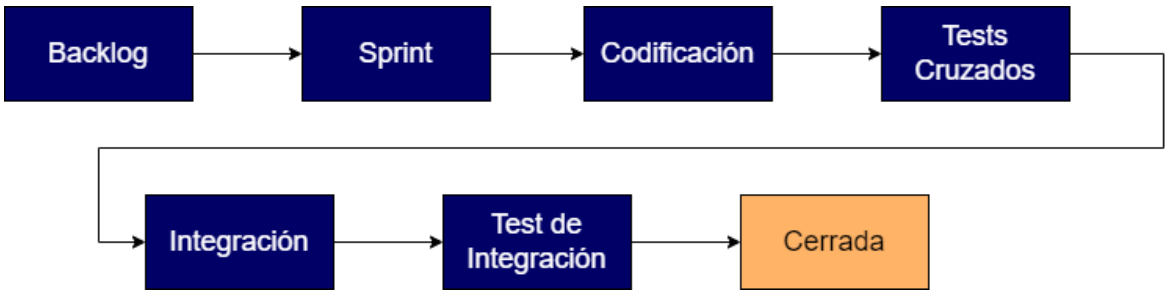


Figura 4.2: Flujo de Trabajo

Las tareas comienzan en el **backlog**, donde están priorizadas, y de donde se sacan para añadirlas a los Sprints. Una vez una tarea está dentro del Sprint, cuando el desarrollador quiere comenzar a realizarla, esta pasa a la etapa de **codificación**, donde se crea la funcionalidad. Una vez se ha creado el código, se hace una *Pull Request* que los demás compañeros de desarrollo pueden revisar. Una vez se ha revisado, si alguno tiene alguna mejora que comentar la comenta. En caso de que todo este bien, cuando la tarea recibe dos aprobados, puede pasar a la etapa de **tests cruzados**, donde se comprueba por parte del *tester* que la funcionalidad desarrollada es la correcta, en cuyo caso, pasará a la etapa de **integración**, donde se integrará a la rama *develop*.

Una vez la integración se ha completado, la tarea pasa a **Test de Integración**, donde se comprobará si se ha creado algún *bug*. Si finalmente no se encuentra ningún *bug* y la funcionalidad funciona correctamente, la tarea pasará a estar **cerrada**.

4.3. MARCO TECNOLÓGICO

A continuación, de modo general, se introducen los detalles más relevantes de las distintas tecnologías utilizadas durante el desarrollo. Si bien, a medida que se vayan utilizando se explicarán más a fondo en los apartados pertinentes.

4.3.1. PHP

PHP es uno de los lenguajes de programación más usados actualmente en el desarrollo web. Corresponde al acrónimo recursivo “PHP: Hypertext Preprocessor” y es usado para escribir código que se ejecuta en el lado del servidor.

Para atender a las peticiones HTTP (Hypertext Transfer Protocol) de una forma sencilla tanto de aprender como de ejecutar, se utiliza Laravel. Un *framework* muy potente que permite crear aplicaciones web desde cero, del que existe mucha documentación y una comunidad grande.

Por último, cabe mencionar Eloquent, un ORM (Object–relational mapping) de Laravel que permite interactuar fácilmente con la base de datos, simplificando las consultas.

4.3.2. React

Para el frontend, React es una solución moderna y muy versátil. Es una librería de JavaScript para construir interfaces de usuario. Su potencia radica en la existencia de componentes, elementos que se construyen una vez y luego se pueden reutilizar en toda la interfaz de una forma muy sencilla. Estos componentes pueden tener, o no, estado, y reaccionar dinámicamente al cambio de este. Además, una gran comunidad respalda la tecnología, creando muchos de estos componentes que se pueden instalar fácilmente.

Desde la React podemos crear toda la lógica que permita navegar por las diferentes pestañas y menús de la aplicación, realizar peticiones HTTP a nuestro backend y tanto realizar como empotrar métricas, entre muchas otras cosas.

4.3.3. MongoDB

MongoDB es sistema de bases de datos NoSQL, concretamente, documental, posicionado, como se puede ver en la figura mostrada en la sección 3.1 (ver Figura 3.2) como el quinto más popular y el primero entre las bases de datos NoSQL. Es una base de datos distribuida que utiliza clusters para mantener la información disponible y replicada. Cuenta con bastantes herramientas, entre las que destacan MongoDB Compass para hacer consultas, agregaciones, y visualización de datos, MongoDB Atlas para administrar los clusters alojados en la nube, y Mongo Charts, que permite crear gráficas utilizando los datos existentes dentro de MongoDB Atlas.

Como hemos mencionado antes, MongoDB se organiza en clusters, dentro de ellos, se encuentran las bases de datos, y dentro de las bases de datos se introducen las colecciones, que es donde finalmente se encontrarán los documentos (ver Figura 4.3). Así mismo, se pueden crear roles que permitan crear, modificar y eliminar todos los elementos antes mencionados.

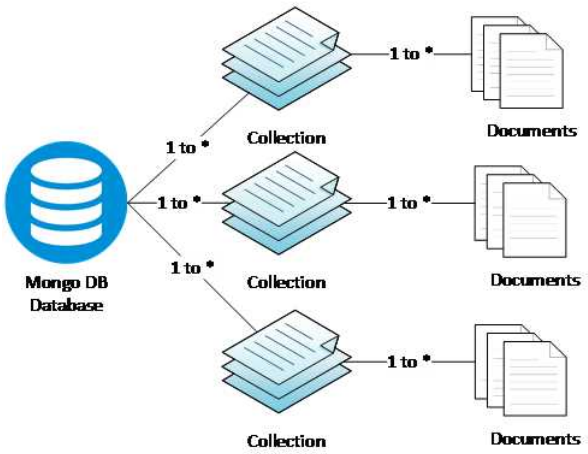


Figura 4.3: Jerarquía MongoDB

Es un sistema de bases de datos muy completo que, además, continúa innovando y

ofreciendo a los desarrolladores nuevas herramientas, como las colecciones *Timeseries* de las que hablaremos más adelante.

4.3.4. Jenkins

Jenkins es un servidor para la integración continua que trabaja junto con Bitbucket, el repositorio de código, para gestionar los cambios que se produzcan en este. Ante cualquier cambio en repositorio, Jenkins ejecuta las tareas que tenga definidas, como crear la build del código, correr los tests y ejecutar el SonarQube. En caso de fallo, notificaría a los desarrolladores para que actúen en consecuencia.

La integración continua (ver Figura 4.4) consiste en la automatización de ciertos procesos que verifican el código, para crear un ejecutable si el código ha pasado tanto los tests como los criterios de calidad definidos.

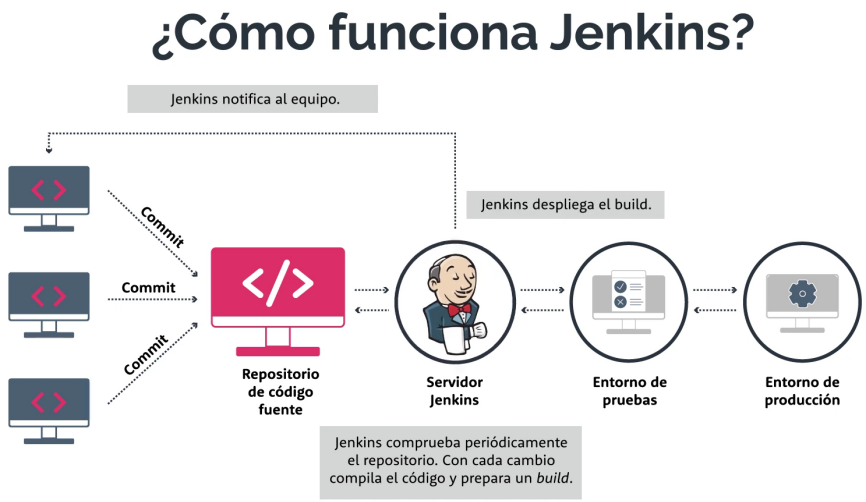


Figura 4.4: Arquitectura de Jenkins

4.3.5. Bitbucket

Bitbucket es un sistema de control de versiones donde se puede usar tanto Git como Mercurial. En este software, los usuarios pueden realizar las tareas básicas de cualquier sistema de control de versiones, como clonar un repositorio, crear ramas, hacer pull requests, etc. Así como integrarlo con software como Jenkins

Una de las cosas más interesantes de Bitbucket es que permite hacer revisiones de código, donde los demás desarrolladores ven el código que se ha subido y lo aprueban o rechazan en función de los posibles errores que tenga este.

4.3.6. Docker

Docker es un sistema de virtualización que permite desplegar código empaquetando este en unidades estandarizadas llamadas contenedores, que incluyen todo lo necesario para que el

software se ejecute correctamente (ver Figura 4.5). Sobre docker se pueden ejecutar comandos que inician o paran los contenedores, comprueban el estado en el que se encuentran, o lanzan peticiones al software que se ha virtualizado para instalar nuevas dependencias, ejecutar tests, etcétera.

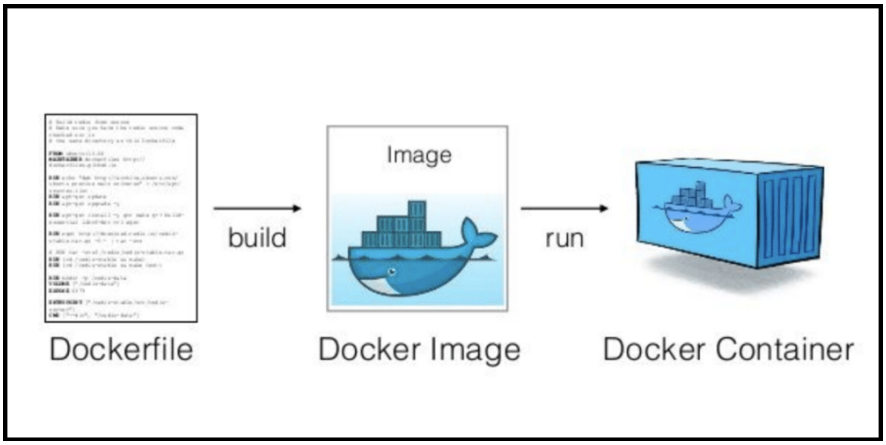


Figura 4.5: Jerarquía de Docker

4.3.7. Redux

Redux es una librería de JavaScript utilizada para contener el estado de las aplicaciones de una forma consistente y predecible. Esta tecnología maneja el estado por medio de los denominados "miniestados", que son subdivisiones del estado principal, y que se crean crean y manejan gracias a los *Reducers*.

Redux puede realizar peticiones tanto síncronas como asíncronas. Las síncronas son las que contienen los datos que queremos dentro del estado, y las asíncronas las que necesitan esperar a que este se cargue mediante algún evento como puede ser una llamada a la API.

Como se puede observar en la figura 4.6, el flujo de *Redux* comienza con un componente de *React* intentando obtener unos datos del estado (1). En caso de este no contenerlo, o de contenerlo pero haber cambiado (2), se lanza una acción (3) esta acción, en caso de necesitar una llamada asíncrona, recurre al *Saga (4)* que se encarga de contactar con el *Provider* (5) el cual ejecutará la llamada a la API y devolverá el resultado (6). Cuando el Saga detecte que le llega el resultado de la llamada, este invocará otra acción (7) que dirá si se ha podido o no obtener los datos de forma correcta. En caso de haberse podido obtener lo que queremos, la acción hará que el *Reducer* almacene los datos de la respuesta que el desarrollador le indique en el estado (9). Por último, El estado notificará al componente y este se actualizará.

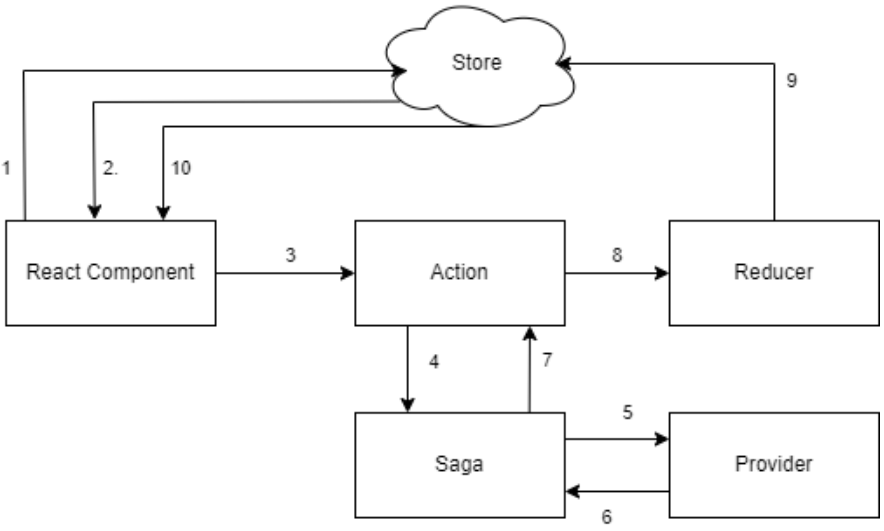


Figura 4.6: Secuencia de acciones de Redux

CAPÍTULO 5

Resultados

En esta sección se repasan las historias de usuario completadas durante cada *Sprint*. Durante el *Sprint* 0 se explicará el proceso de entrada a la empresa y configuración de los elementos necesarios para el desarrollo, y durante los sucesivos *Sprints* y con el objeto de mantener un estilo homogéneo, se describirán las historias de usuario completadas mediante una tabla con los siguientes elementos:

- **Nombre historia**: Nombre que identifica la historias de usuario.
- **Número**: Identificador de la historias de usuario.
- **Esfuerzo**: Estimación realizada en el *planning poker* de la cantidad de esfuerzo necesario para completar la tarea.
- **Descripción**: Indicador de lo que se quiere obtener con la historia.
- **Definición de Hecho**: Condiciones que debe cumplir la tarea para considerarse finalizada.

5.1. SPRINT 0

Este primer Sprint fue utilizado para la presentación del equipo y puesta en marcha del proyecto.

Primero de todo, se configuró el entorno de desarrollo en el ordenador de trabajo. La configuración constó de obtener acceso a una VDI (*Virtual Development Infrastructure*), que es un entorno Windows en remoto, útil para que todos los desarrolladores tengan la misma configuración y no existan problemas derivados de disparidades en esto. Dentro de la VDI, se dio acceso a las bases de datos necesarias, se generó la conexión con el IDE (*Integrated Development Environment*) y se cambió la configuración de la terminal para permitir acceder a los entornos donde las aplicaciones están desplegadas.

Una vez configurado todo esto, se analizó la aplicación de SIMBAD para ver qué métricas contenía y cómo estaba organizada, así como el código para la familiarización con Laravel, React, y el flujo de datos que había en la aplicación. Además, también se generaron los

usuarios y se recibieron los permisos pertinentes para poder leer y escribir de las bases de datos necesarias.

A continuación, se describieron y agruparon una serie de métricas para añadirlas a la pila de producto, donde más tarde se estimarían siguiendo la técnica de búsqueda de consenso de *Planning Poker*.

Por último, se creó la arquitectura de la aplicación con cada uno de sus componentes integrado y funcional (ver figura 5.1). La arquitectura de la aplicación contiene dos bases de datos (MariaDB GlobalSuite y MongoDB GlobalSuite) de las que la API obtiene información periódicamente. Esta periodicidad la aporta el planificador (*Scheduler*) que invoca comandos cada cierto tiempo si así se le indica. Estos comandos, aparte de ser los encargados de obtener la información de las dos bases de datos ya mencionadas, son también los responsables de almacenarlos, previa transformación, en una tercera base de datos llamada MongoDB Cronos. Esta base de datos, MongoDB Cronos, está alojada en el servicio base de datos en la nube MongoDB Atlas, que aporta algunas herramientas muy útiles como MongoCharts, que permite crear gráficas que luego se pueden empotrar dentro de la aplicación gracias a un SDK. La aplición de React actuaría como interfaz, esta se comunica con MongoCharts por medio del antes mencionado SDK, y coloca las gráficas en la interfaz.

Una vez listo esto, queda preparado el entorno para comenzar el desarrollo.

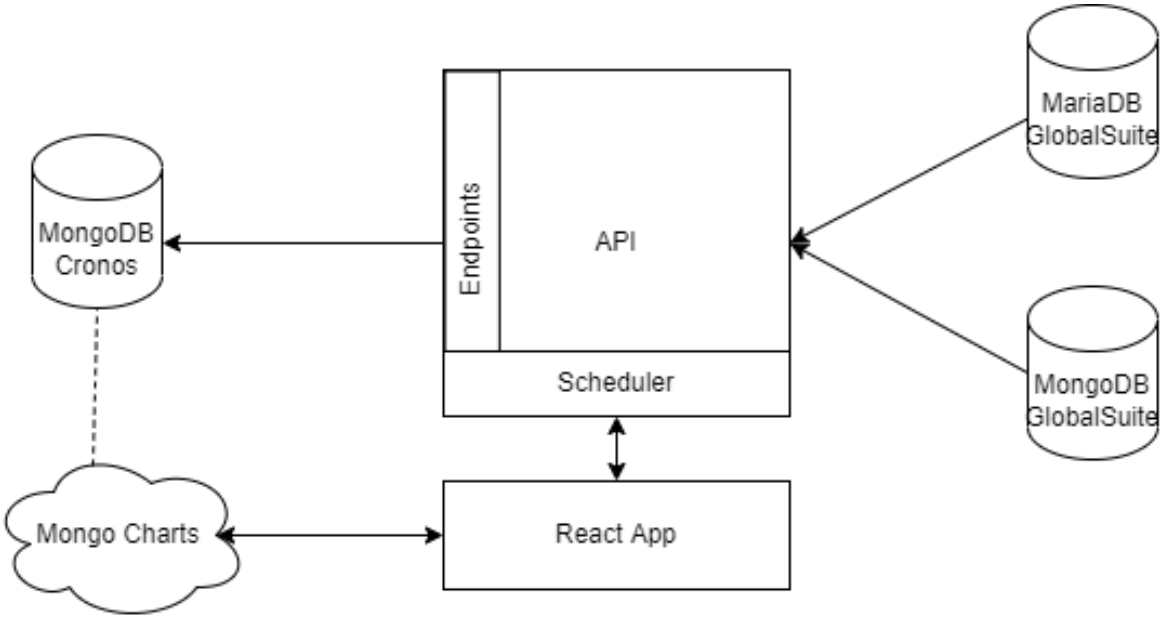


Figura 5.1: Arquitectura de la aplicación

5.2. SPRINT 1

El objetivo de este Sprint fue poder ver la primera gráfica dentro de la aplicación web, y para lograrlo, se realizó la tarea descrita en la tabla 5.1.

Tabla 5.1: Sprint 1 - Actividad BBDD - Tamaño

Historia de usuario	
Nombre historia: Actividad BBDD - Tamaño	
Número: 1	**Esfuerzo**: 13
Descripción: Como usuario quiero ver datos sobre la evolución temporal del tamaño de las bases de datos	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade una nueva pestaña a la aplicación web llamada *Actividad BBDD* que contenga dos gráficas, una que indique el tamaño de los esquemas en MariaDB y otra del tamaño de los esquemas en MongoDB. Además, estas dos gráficas deben ser filtrables por fecha y por esquema.	

El desarrollo del Sprint fue como se esperaba, un comienzo marcado por la poca experiencia con PHP que creó dificultades, pero que fue desapareciendo conforme se trabajaba en la funcionalidad. Así mismo, se realizó y se obtuvo el certificado del curso de *MongoDB University* llamado *M001 - MongoDB Basics*[1], que aporta una visión general de todo lo necesario para trabajar con esta tecnología.

Como eventos relevantes del sprint, cabe destacar que se estudió si merecía la pena innovar en cuanto a la implementación de un nuevo tipo de colección existente en MongoDB (lanzado en la versión 5.0.), llamado *Time Series Collection*[2] que destaca por su orientación al trabajo con medidas tomadas durante un periodo de tiempo. Estas colecciones contienen un campo obligatorios, llamado *TimeField*, y dos opcionales llamados *metaField* y *granularity* que aportan, el primero la posibilidad de crear campos indexables, y el segundo un TTL (*Time To Live*) a los datos (ver Listado 5.1), interesante si no nos interesa almacenarlos permanentemente. Además, a este tipo de colecciones se les puede añadir cualquier otro campo con otro si no necesitamos que sea indexable.

Tras el estudio de las principales ventajas e inconvenientes, desarrollamos una lista para poder comparar la opción de implementar colecciones normales, o colecciones *Time Series* opciones:

Ventajas

- Existe amplia documentación.
- Colecciones y lecturas de 3 a 5 veces más comprimidas y rápidas.[3]
- Laravel acepta la inserción de colecciones del tipo *Time Series* usando modelos.
- Es una tecnología emergente que se adapta a nuestros objetivos, ya que queremos registros temporales.

[1]https://university.mongodb.com/courses/M001/about
[2]https://www.mongodb.com/docs/manual/core/timeseries-collections/
[3]https://www.mongodb.com/developer/products/mongodb/paginations-time-series-collections-in-five-minutes

- Las colecciones *Time Series* tienen más tipos de agregaciones que las normales, como las derivadas o integrales.

Inconvenientes:

- Es más complejo editar o modificar datos que en una colección normal
- La creación de índices solo se puede hacer sobre los campos descritos dentro de la matriz *Metadata*, lo que dificulta el diseño de las colecciones.
- Incertidumbre por falta de conocimiento previo en el uso de colecciones de este tipo.

Teniendo en cuenta las ventajas e inconvenientes, decidimos que era interesante acceder a implementar este tipo de colecciones.

Listado 5.1: Ejemplo de creación de coleccion *Time Series*

```
db.createCollection(
    "weather",
    {
       timeseries: {
          timeField: "timestamp",
          metaField: "metadata",
          granularity: "hours"
       }
    }
)
```

A continuación, se desarrollo el comportamiento de la API, para lo cual se creó un fragmento de código que se conectaba con la base de datos fuente para ejecutar dos consultas, cuyos datos serían procesados para ser insertables en la base de datos de MongoDB mediante un modelo de Eloquent, haciendo uso de este ORM. Una vez se probó que esto funcionaba, se creó un comando para que la inserción se ejecutase automáticamente cada cierto tiempo, aunque esto no se probó puesto que en el entorno de desarrollo local no está configurada la funcionalidad.

Para finalizar el desarrollo, para mostrar las gráficas, utilizamos el SDK de Javascript de Mongo Charts para empotrar gráficas en nuestra aplicación de React y creamos el código necesario para hacer que estas pudieran filtrarse por fecha y por esquema.

Lo último que se hizo en el Sprint fue lo conocido como *Code Review* o Revisión de Código, que consiste en la creación de una *pull request* en la que se pone a los demás compañeros del departamento de desarrollo como revisores y ellos comprueban que el código sea de calidad, mantenible, y no tenga *Code Smell*. Aquí salieron algunos comentarios sobre el código que se corrigieron, sobre todo relativos a peculiaridades de PHP como el uso de las funciones *array_merge*, *array_walk*, *array_filter* o *array_reduce* en vez de bucles *for*.

5.3. SPRINT 2

El objetivo de este Sprint fue poder añadir las gráficas descritas en la tarea 5.2 en su correspondiente pestaña.

Tabla 5.2: Sprint 2 - Uso GS - Empresas - Datos Empresas

Historia de usuario	
Número: 2	**Esfuerzo**: 8
Nombre historia: Uso GS - Empresas - Datos Empresas	
Descripción: Como usuario quiero ver datos sobre la evolución temporal de datos sobre estados y tipos de empresa	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade una nueva pestaña a la aplicación web llamada *Uso GS* que contenga dos gráficas, una que contenga las empresas por estado y otra que contenga las empresas por tipo. Además, estas dos gráficas deben ser filtrables por fecha	

Durante el *Sprint*, se asistió a un curso impartido por MongoDB sobre *Time Series* en el se daba una visión general sobre esta nueva categoría de colecciones, además de mostrar algunas de las mejores formas de estructuras los datos dentro de los documentos. La técnica de estructurar documentos que vimos oportuna utilizar fue la de *bucketing* (ver figura 5.2), consistente en la agrupación de los datos según alguna de sus características.

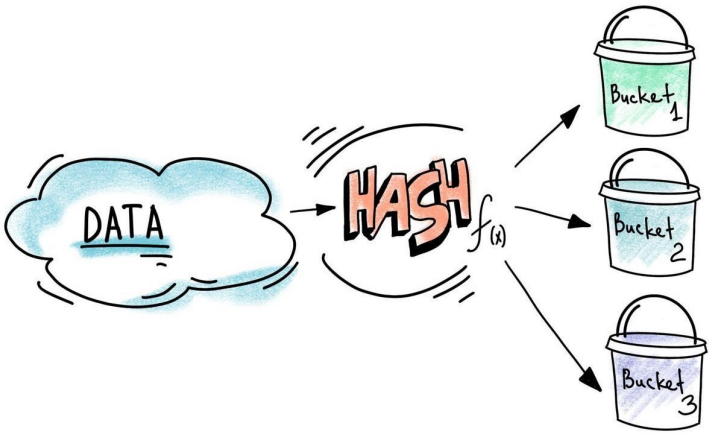


Figura 5.2: *Bucketing*

Decidimos entonces que, donde fuera posible, agruparíamos los datos por esquema de base de datos como se muestra en el ejemplo de documento que aparece en el Listado 5.2, teniendo en cuenta además las limitaciones[4] de tamaño que MongoDB pone a sus documentos y colecciones para que la aplicación pudiera escalar en un futuro. Como se puede apreciar, existen los campos *timestamp* y *metadata*, fijos en todas las colecciones hechas en este proyecto, y dentro del campo *metadata* encontramos un esquema junto con los valores que queremos almacenar, en este caso, una matriz con dos elementos.

[4]https://www.mongodb.com/docs/manual/reference/limits/

Listado 5.2: Ejemplo de documento con Bucketing

```
timestamp: 2022-06-07T08:08:12.044+00:00
metadata: {
    schema: esquema1
    valores:[
      [
          campo1: valor1
          array1:
                [
                    [elemento0]
                    [elemento1]
                ]
      ]
      ,
      [
          campo2: valor2
          array2:
              [
                    [elemento0]
                    [elemento1]
              ]
      ]
    ]
}
```

Queriendo aplicar la técnica de *bucketing*, para la realización de la tarea 5.2, surgió el problema de que obteníamos datos de varias consultas, por tanto, había que unirlos de alguna forma para introducirlos en el mismo documento. Tras investigar la documentación de PHP, se encontró la existencia del método *array_merge_recursive()*[5], que cuando toma varios diccionario con pares clave-valor, une los valores cuyas claves coinciden en una matriz (Ver Listado 5.3).

[5]https://www.php.net/manual/es/function.array-merge-recursive.php

Listado 5.3: Ejemplo uso de array_merge_recursive

```
$array1 = [
    "clave1" => "valor1_clave1",
    "clave2" => "valor2"
    ];

$array2 = [
    "clave1" => "valor1_clave2",
    "clave3" => "valor3"
    ];

$mergedArrays = array_merge_recursive($array1, $array2);

/* El valor de $mergedArrays sería:
*   ["clave1" => ["valor1_clave1", "valor1_clave2"]],
*   ["clave2" => "valor2"],
*   ["clave3" => "valor3"]
*/
```

Como nos interesaba iterar sobre las claves del diccionario generado a la hora de hacer un *array_map()*[6], la fórmula descrita en el Listado 5.4 no nos era útil ya que itera sobre los valores de *$mergedArrays*, no sobre sus claves. Por ello, usando la variante descrita en el Listado 5.5 podemos iterar por las claves mediante la utilización de *array_keys($mergedArrays)*[7]. Como también nos es de interés acceder a los valores de *$mergedArrays*, utilizamos *use ($mergedArrays)* para poder utilizar esta variable dentro del código de la función usando *$mergedArrays[$iterableElement]*. El resultado queda plasmado en la figura 5.3.

Listado 5.4: Ejemplo uso de array_map_no_complaciente

```
array_map(
    function ($iterableElement) {
        //Codigo
    },
    $mergedArrays
);
```

Listado 5.5: Ejemplo uso de array_map_complaciente

```
array_map(
    function ($iterableElement) use ($mergedArrays) {
        //Codigo
    },
    array_keys($mergedArrays)
);
```

[6]https://www.php.net/manual/es/function.array-map.php
[7]https://www.php.net/manual/es/function.array-keys.php

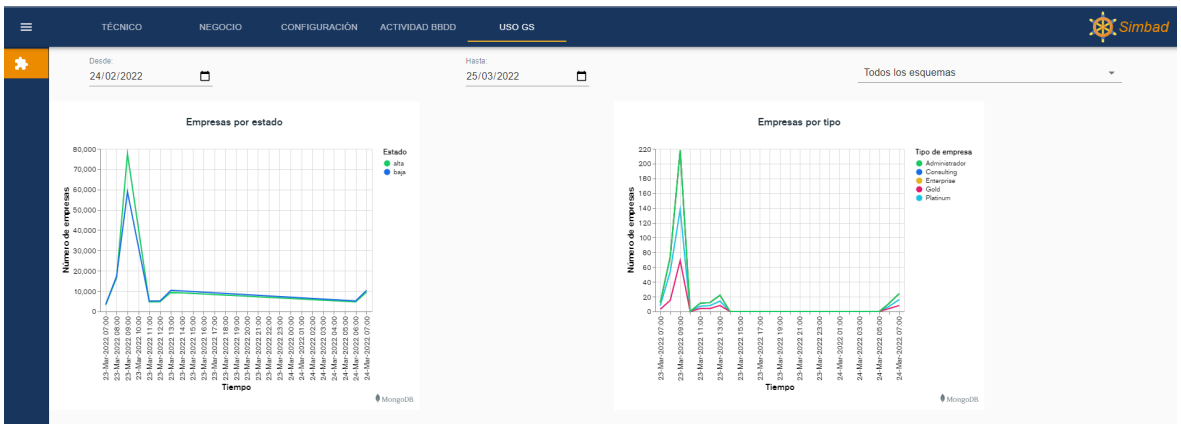


Figura 5.3: Gráficas asociadas a la tarea 5.2

Por último, cabe destacar que durante este *Sprint* se creó una clase *Trait* destinada a realizar las llamadas a los métodos de *Laravel* que realizan las peticiones a la base de datos, y devolver la respuesta. Un *Trait* es un mecanismo de reutilización utilizado en lenguajes con herencia simple que permite reutilizar conjuntos de métodos de varias clases. Los métodos de los *Traits* deben ser todo lo reutilizables que se pueda.

5.4. SPRINT 3

El objetivo del tercer Sprint fue añadir, en una pestaña llamada Empresas, información sobre Usuarios, Empresas y Licencias, lo cual se consiguió realizando las tareas descritas en las tablas 5.3, 5.4 y 5.5

Tabla 5.3: Sprint 3 - Uso GS-Empresas-Usuarios

Historia de usuario	
Número: 3	**Esfuerzo**: 5
Nombre historia: Uso GS-Empresas-Usuarios	
Descripción: Como usuario quiero ver datos sobre la evolución temporal del total de usuarios y de los usuarios por tipo	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade a la pestaña *Uso GS* una nueva sección donde aparezcan dos gráficas, una que contenga los usuarios que hay de cada tipo, y otra que muestre el total de usuarios que existen en la aplicación. Además, estas dos gráficas deben ser filtrables por fecha y por esquema.	

Para la tarea 5.3 se creó una nueva pestaña llamada “Usuarios”, también se creó el componente que contiene las dos gráficas y se añadió el componente en la pestaña. En cuanto a la API, su desarrollo fue como se esperaba: Primero, se crea la consulta SQL que nos devuelve los datos que necesitamos; segundo, se crea el modelo que se usará para insertar los datos; tercero, se crean los tests que deberá cumplir la clase; cuarto, se crea la clase que maneje los datos y haga la petición de inserción al modelo; quinto, se crea el comando y se programa su ejecución. Después de realizar esto, se prueba el comando y se comprueba que los datos que se han insertado en la tabla son los que se esperaba que se insertasen.

Tabla 5.4: Sprint 3 - Uso GS-Empresas-Licencias

Historia de usuario	
Número: 4	**Esfuerzo**: 5
Nombre historia: Uso GS-Empresas-Licencias	
Descripción: Como usuario quiero ver datos sobre la evolución temporal de las licencias de alta y módulos más usados	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade a la pestaña *Uso GS* una nueva sección donde aparezcan dos gráficas, una que contenga las licencias de alta más usadas y otra que muestre los módulos de la aplicación más usados. Además, estas dos gráficas deben ser filtrables por fecha y por esquema.	

La tarea 5.4 también fue conforme a lo planificado, de ella destaca que hubo una refactorización en la forma de llamar a los comandos, que en vez de seguir la estructura definida en el Listado 5.6, pasaron a definirse como se muestra en el Listado 5.7 para reflejar mejor su propósito.

Listado 5.6: Antigua estructura del comando

```
Check:LoadData<nombre>Command
```

Listado 5.7: Nueva estructura del comando

```
measurement:LoadData<nombre>Command
```

Tabla 5.5: Sprint 3 - Uso GS-Empresas-Licencias y empresas

Historia de usuario	
Número: 6	**Esfuerzo**: 3
Nombre historia: Uso GS-Empresas-Licencias y empresas	
Descripción: Como usuario quiero ver datos sobre la evolución temporal de el número de licencias por empresa	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade a la aplicación una gráfica que contenga las licencias más usadas por empresa. Además, estas dos gráficas deben ser filtrables por fecha y por esquema.	

La realización de la tarea 5.5 se vio acortada por la posibilidad de reutilizar los datos de la colección creada en la tarea 5.4, por tanto, solo fue necesario crear la gráfica en MongoCharts y empotrarla en la pestaña de 'Empresas'. Además se movió la pestaña de configuración a la derecha del todo, que por experiencia de usuario, era mejor tenerla la última que la tercera.

El Sprint se desarrolló con normalidad, dado que la forma de trabajar y los conceptos estaban asentados. El resultado de las tareas se puede ver en las figuras 5.4, 5.5 y 5.6.

En la revisión de código no se encontró ningún error relevante y se pudo hacer *merge* de la rama de trabajo a la de *develop* correctamente.

Por último, se completó el curso *M103: Basic Cluster Administration* [8] de MongoDB para

[8]https://university.mongodb.com/mercury/M103/2022_February_8/overview

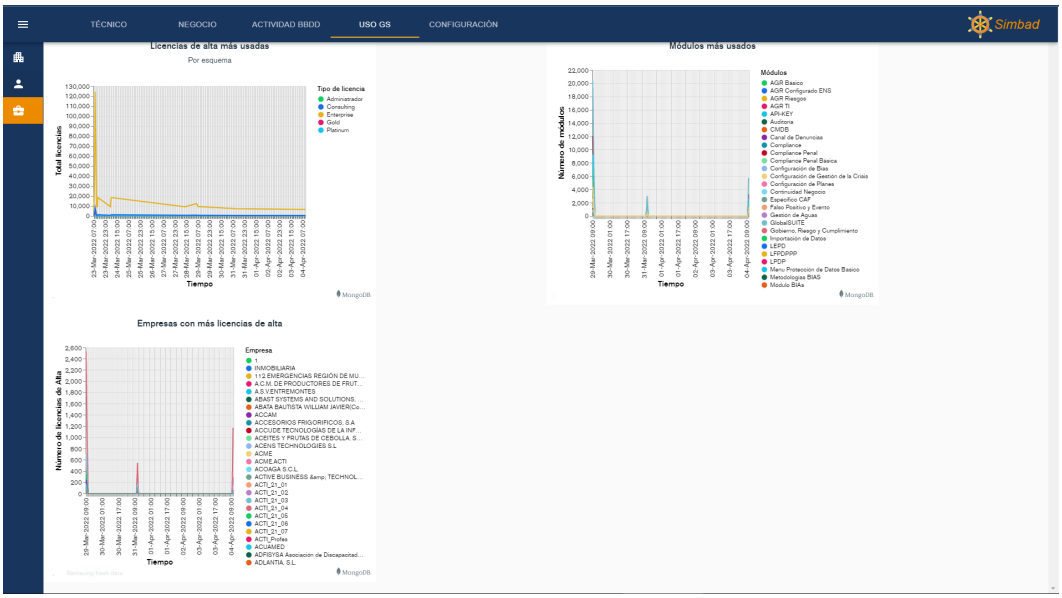


Figura 5.4: Pestaña resultado de la tarea 5.4

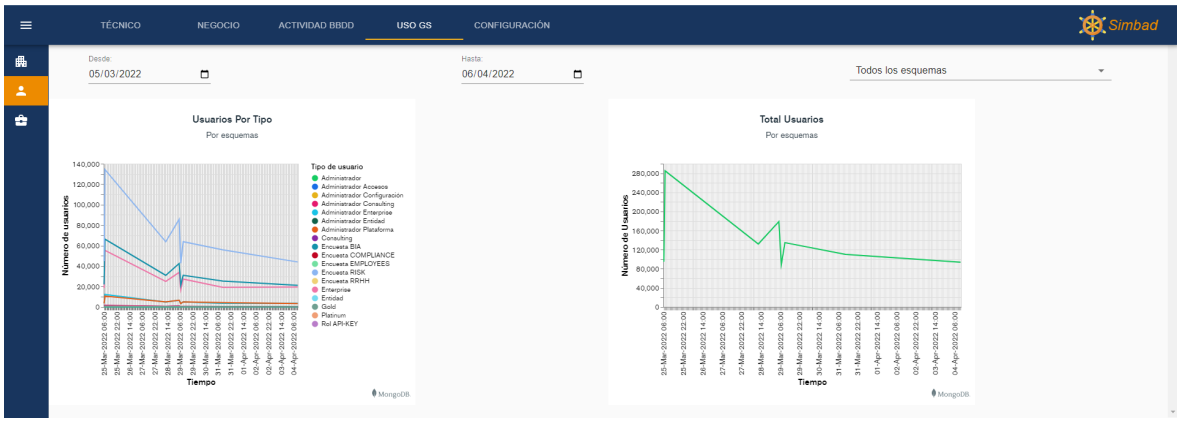


Figura 5.5: Pestaña resultado de la tarea 5.3

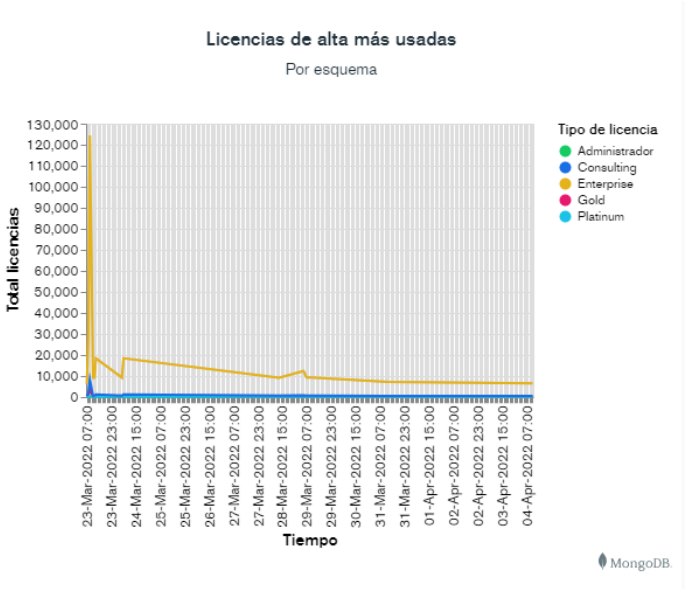


Figura 5.6: Pestaña resultado de la tarea 5.5

tener una visión general sobre administración de *Clusters* que va desde como configurar e iniciar una instancia de MongoDB, hasta conceptos más avanzados de replicación y fragmentación de datos.

5.5. SPRINT 4

El objetivo del cuarto Sprint fue la adición de gráficas relativas a encuestas y tamaño de las tablas así como la activación del *scheduler* en los entornos pertinentes. El trabajo realizado en el sprint queda registrado en las tareas descritas en las tablas 5.6 y 5.7.

Tabla 5.6: Sprint 4 - Actividad BBDD-Tablas y esquemas-Tamaño

Historia de usuario	
Número: 7	**Esfuerzo**: 8
Nombre historia: Actividad BBDD-Tablas y esquemas-Tamaño	
Descripción: Como usuario quiero ver datos sobre la evolución temporal del tamaño de las tablas almacenadas	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade a la aplicación la pestaña *Actividad BBDD* y esta contiene dos gráficas con información del tamaño de las tablas de MariaDB y MongoDB respectivamente. Además, estas dos gráficas deben ser filtrables por fecha y por esquema.	

Tabla 5.7: Sprint 4 - Uso GS - Encuestas - encuestas por tipo

Historia de usuario	
Número: 8	**Esfuerzo**: 5
Nombre historia: Uso GS - Encuestas - encuestas por tipo	
Descripción: Como usuario quiero ver datos sobre la evolución temporal de datos sobre número, estado, tipo y fecha de encuestas.	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añaden cinco gráficas, que muestren las encuestas por tipo, las empresas con más encuestas, las encuestas finalizadas, encuestas consolidadas y encuestas por fecha de lanzamiento. Además, estas dos gráficas deben ser filtrables por fecha y por esquema. Además, estas dos gráficas deben ser filtrables por fecha y por esquema.	

El *Scheduler* es el encargado de ejecutar cada cierto tiempo, estipulado por el programador, un comando que realice una acción. En nuestro caso, la acción a realizar es la llamada a los métodos encargados de añadir información a la base de datos. Al desplegar en los entornos en los que debía estar configurado esta funcionalidad, nos dimos cuenta de que no funcionaba. Tras configurar las máquinas donde se despliega la aplicación, repasar el flujo del código, y hacer algunos cambios en este logramos que funcionase y se comenzaron a añadir los datos automáticamente todos los días a las tres de la madrugada, ya que a esta hora la afluencia de tráfico en la aplicación era menor que en el horario laboral.

Además de los trabajos con el *Scheduler*, como se muestra en las figuras 5.7 y 5.8, la estética de la aplicación cambió para adecuarse a la nueva estética de GlobalSuite, y las

gráficas se mejoraron para mostrar los puntos de datos, integrarlas mejor con la aplicación, y quitar el logo de MongoDB para dejarlas más limpias.

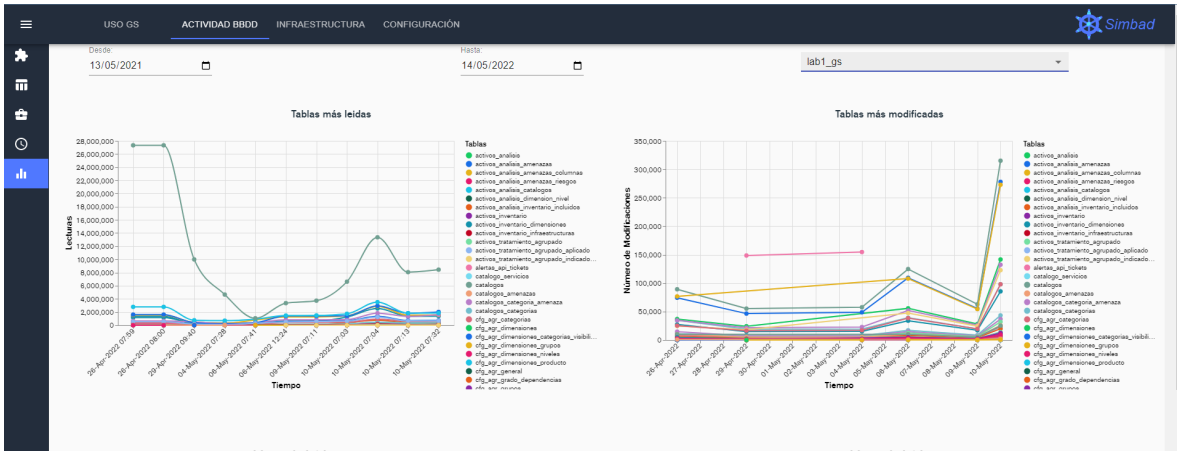


Figura 5.7: Pestaña resultado de la tarea 5.8

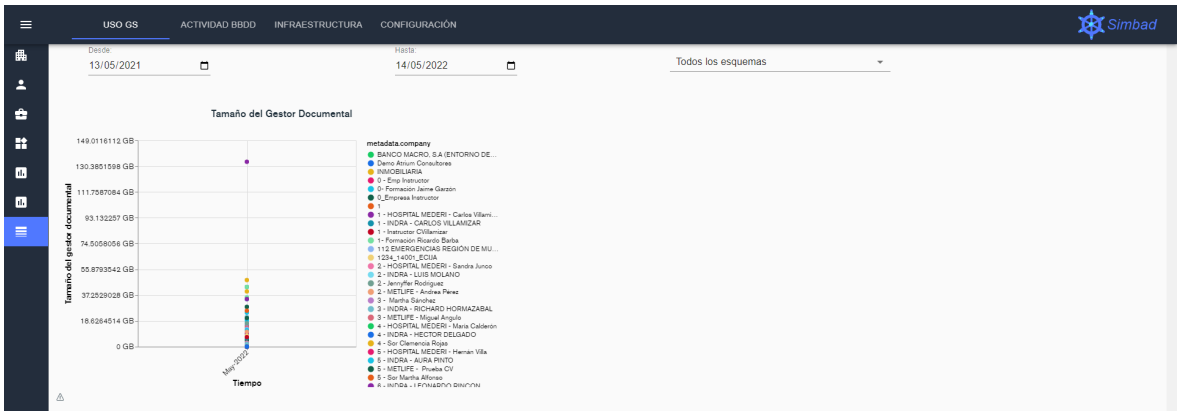


Figura 5.8: Pestaña resultado de la tarea 5.9

Por último, abordamos un problema que surge de la existencia de varios entornos, y es que los datos no son los mismos en el entorno de producción que en los entornos de desarrollo o pruebas, y por tanto, las gráficas a mostrar también son distintas. La solución que dimos fue la creación de una colección dentro de cada base de datos (que cambia según el entorno) para contener ahí los identificadores de las gráficas a mostrar. Desde la interfaz realizaríamos una petición, y esta nos devolvería el identificador de la gráfica a mostrar. Durante este *Sprint* se pudo realizar lo explicado en este párrafo correctamente.

5.6. SPRINT 5

El objetivo de este *Sprint* fue la integración de las gráfica con datos sobre lecturas y modificaciones de tablas, así como las del tamaño del gestor documental.

El desarrollo fue con normalidad y se realizaron las tareas de las tablas 5.8 y 5.9, pero a la hora de la revisión de código se detectaron ciertos problemas a resolver. El primero, que había un fragmento de código que se repetía mucho, dedicado a contener los selectores

de fecha y el selector de esquemas, y debía ser extraído, y el segundo, que se realizaban peticiones a la API directamente desde los componentes que se dedicaban a mostrar la información, lo cual no desacoplaba suficientemente los componentes.

Tabla 5.8: Sprint 5 - Actividad BBDD - Tablas y esquemas - Leídas y modificadas

Historia de usuario	
Número: 9	**Esfuerzo**: 5
Nombre historia: Actividad BBDD - Tablas y esquemas - Leídas y modificadas	
Descripción: Como usuario quiero ver datos sobre la evolución temporal de la cantidad de lecturas y modificaciones sobre las tablas	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añaden en la pestaña de *Actividad BBDD* dos gráficas que muestren, una las tablas más leídas y otra las más modificadas de MariaDB. Además, estas dos gráficas deben ser filtrables por fecha y por esquema.	

Tabla 5.9: Sprint 5 - Uso GS - Gestor Documental - tamaño gestor

Historia de usuario	
Número: 10	**Esfuerzo**: 5
Nombre historia: Uso GS - Gestor Documental - tamaño gestor	
Descripción: Como usuario quiero ver datos sobre la evolución temporal del tamaño del gestor documental (Herramienta de GlobalSuite)	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade en la pestaña de *Uso GS* una gráfica que muestre, el tamaño del gestor documental de cada sistema de gestión gráficas deben ser filtrables por fecha y por esquema.	

El problema del código se resolvió, y se creó un componente externo que aglutinaba tanto el selector de fecha como el selector de esquema, esto redujo el código duplicado en todos los componentes, las lineas de código necesarias para crear la funcionalidad y mejoró la legibilidad de código. Así mismo, también se extrajo la lógica de creación de filtros para las gráficas, que también venía repitiéndose en las anteriores tareas. Aplicando los conceptos adquiridos en el curso de *React* que la empresa proporcionó a los desarrolladores, se consiguió implementar lo mencionado utilizando componentes funcionales, tendencia que *React* recomienda seguir.

En cuanto al desacoplamiento de las llamadas a la API, contemplamos *Redux* como una solución correcta. *Redux* se encargaría de realizar las peticiones a la API y de mantener el estado almacenado en la aplicación para cuando se quiera consultar. Esto desacopla la aplicación y evita repetir llamadas a la API. Esto lo realizamos durante la tarea 5.8, tarea en la que decidimos que abordaríamos la refactorización de código que controla el problema de tener varios entornos donde desplegar la aplicación como se explica al final de la sección 5.6. Al usar *Redux* se consiguió que, al cargar un componente, se realizaría una petición a la API que devolviera el identificador de la gráfica, para después pasárselo como parámetro al componente que pinta el gráfico, y así se mostraría el del entorno que hay desplegado.

Para conseguir esto utilizando *Redux*, se creó un Reducer con un miniestado compuesto por los identificadores de las gráficas a mostrar. También se crearon las acciones de *Redux* pertinentes para la comunicación de eventos, un *Sagas* para controlar las llamadas asíncronas a la API, y un *Provider* para realizar esta llamadas.

En cuanto a los componentes, la aplicación de *Redux* supone añadir dos funciones, una *mapStateToProps* y otra *mapDispatchToProps*, la primera asocia una variable contenida en el miniestado a una variable contenida en las *props* del componente. La segunda asocia una función a una acción de *Redux* para que cuando se llame a la función, se esté llamando a la acción que lanzará el evento obtención de datos. Esta función se llama desde la función de *componentDidMount()* para que se haga cuando el componente se renderice, y por último, a la hora de exportar el componente se debe hacer de la forma *export default connect(mapStateToProps, mapDispatchToProps)(Component);* en vez de *export default Component;*

Esta aplicación de *Redux* supone también algunos cambios en las pruebas, siendo la más relevante la necesidad de simular el miniestado del que el componente toma los datos.

Para continuar con la mejora del código se externalizaron las variables que contenían datos sobre tamaños a un componente nuevo que se importó en los componentes necesarios, eliminando lineas de código y mejorando la mantenibilidad.

Por último, a la hora de crear la *pull request* nos dimos cuenta de que había aumentado la cantidad de cobertura necesaria para integrar el código en la rama *develop*, por lo que se probó lo necesario para cubrir las necesidades. Además, los tests del cliente cambiaron puesto que la implementación de *Redux* así lo precisaba. Gracias a la creación de estos tests, se consiguió casi un 72 % de cobertura en la tarea 5.8 como se puede ver en la figura 5.9, donde resalta a su derecha que también se mantienen los *Code Smell* a 0.

Las pestañas resultantes se muestran en las figuras 5.7 y 5.8, que aparecieron en el apartado anterior.

5.7. SPRINT 6

El objetivo de este *Sprint* fue finalizar el producto a entregar en el trabajo de fin de grado añadiéndole todas las gráficas posibles.

El desarrollo fue con normalidad y se realizaron las tareas de las tablas 5.10, 5.11, 5.12, 5.13, 5.14, 5.14, 5.16, 5.17 y 5.18. Durante este *Sprint* se notó la mejora e interiorización del proceso del desarrollo, esto, sumado a los pocos errores inesperados ocurridos durante este periodo de tiempo, han hecho posible la realización de un gran número de tareas.

Las dos particularidades de este *Sprint* vienen de la mejora de la aplicación. La primera, porque apareció un *bug* durante una de las revisiones periódicas de la aplicación y la segunda porque además de añadir gráficas con registro temporal, se añadieron gráficas que muestran el estado actual de la aplicación.

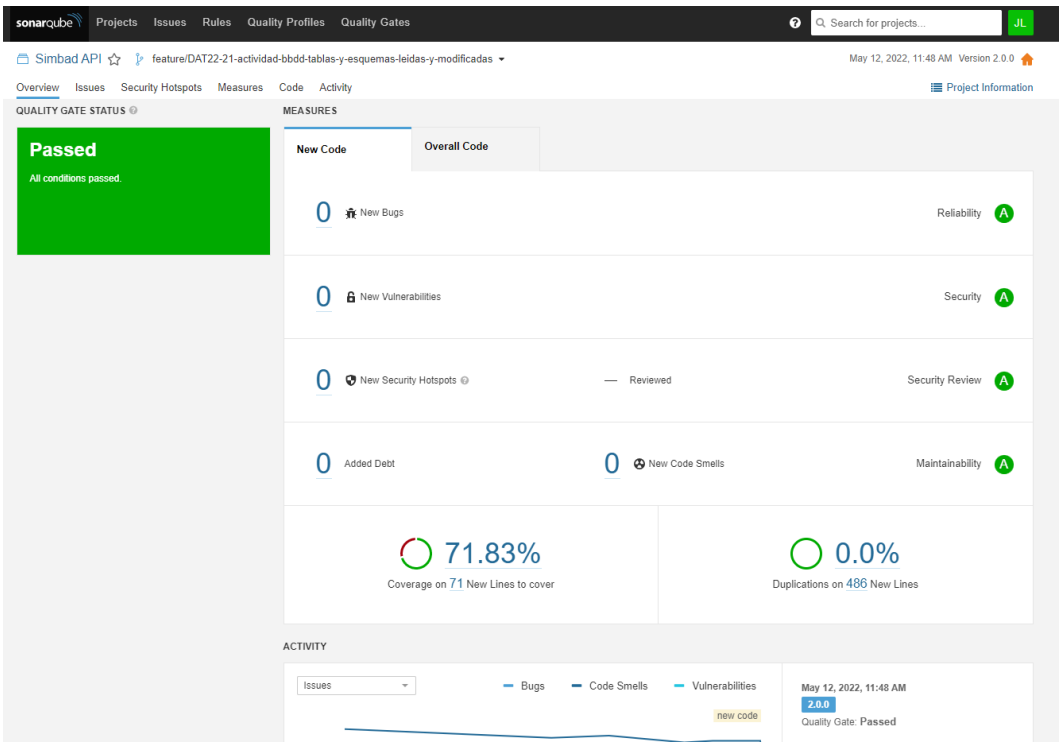


Figura 5.9: Captura de SonarQube de la tarea 5.8

La corrección del *bug* se realizó mediante la tarea 5.17, en ella también se realizaron algunas mejoras gracias a la revisión de código. Estas mejoras iban ligadas al control de fechas. Dado que las fechas de la base de datos están en formato *UNIX*, y las fechas que MongoDB utiliza tienen el formato *UTC* y nosotros queríamos almacenar las fechas *UNIX* con formato *UTC* había que realizar la conversión multiplicando por mil para convertir de segundos a milisegundos. Esto se solucionó creando un *Trait* de control de fechas que contuviese un método que recibe una fecha y realiza la conversión para devolver la fecha en el formato correcto. Por buenas prácticas, el número mil se extrajo a una variable dentro de una clase de constantes para evitar una multiplicación en la que uno de los dos elementos es un número ya que, sin contexto, el número en si no aporta significado.

Tabla 5.10: Sprint 6 - Uso GS - Elementos Generales - activos, servicios y procesos

Historia de usuario	
Número: 11	**Esfuerzo**: 5
Nombre historia: Uso GS - Elementos Generales - activos, servicios y procesos	
Descripción: Como usuario quiero ver datos sobre la evolución temporal de la cantidad de activos, servicios y procesos que cada empresa contiene.	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade en la pestaña de *Uso GS* una gráfica que muestre, tres gráficas: una con el número de activos disponibles, los servicios y los procesos que tiene cada empresa. Además, estas gráficas deben ser filtrables por fecha y por esquema.	

Durante la realización de la tarea 5.10 surgió un problema similar al solucionado en la tarea 5.2, necesitamos del uso de la función *array_merge_recursive()* para unir los valores de

varios diccionarios, pero a la hora de hacerlo, el resultado concatenaba las entradas en vez de combinarlas. Tras una investigación, se detecto que el problema estaba en que las claves de las entradas eran numéricas y no cadenas, esto, según la documentación, concatenaba en vez de combinar, así que en la etapa de tratamiento de datos previa a la combinación, se hizo que estas claves fueran cadenas, y el problema quedó resuelto.

Tabla 5.11: Sprint 6 - Uso GS - Elementos Generales - Scorecard y auditorías

Historia de usuario	
Número: 12	**Esfuerzo**: 3
Nombre historia: Uso GS - Elementos Generales - Scorecard y auditorías	
Descripción: Como usuario quiero ver datos sobre la evolución temporal de la cantidad de métricas, indicadores y auditorías que cada empresa contiene	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade en la pestaña de *Uso GS* una gráfica que muestre, el número de métricas, indicadores y auditorías que cada empresa tiene. Además, estas gráficas deben ser filtrables por fecha y por esquema.	

Tabla 5.12: Sprint 6 - Uso GS - Análisis de Riesgos - Total Análisis y Riesgos

Historia de usuario	
Número: 13	**Esfuerzo**: 5
Nombre historia: Uso GS - Análisis de Riesgos - Total Análisis y Riesgos	
Descripción: Como usuario quiero ver datos sobre la evolución temporal del número de análisis de riesgos realizados por cada empresa, así como del número total de riesgos asociados a estas.	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade en la pestaña de *Uso GS* una gráfica que muestre, para cada empresa, el número de análisis de riesgos y el número total de riesgos. Además, estas gráficas deben ser filtrables por fecha y por esquema.	

Tabla 5.13: Sprint 6 - Uso GS - Tickets - Tickets por Empresa

Historia de usuario	
Número: 14	**Esfuerzo**: 5
Nombre historia: Uso GS - Tickets - Tickets por Empresa	
Descripción: Como usuario quiero ver datos sobre la evolución temporal del número total de tickets por empresa, y de los distintos tipos de tickets que hay en en GlobalSuite	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añade en la pestaña de *Uso GS* una gráfica que muestre, de gráficas, una con el número total de tickets por empresa y otra con los distintos tipos de tickets	

Durante la realización de la tarea 5.14 se añadió en el menú lateral la posibilidad de visualizar una descripción emergente de cada elemento (Ver figura 5.10). Esto aporta usabilidad al producto ya que ahora el usuario no tiene que abrir el menú si quiere saber cuál es el nombre de algunos de los elementos que lo componen, solamente con poner el ratón encima se podrá obtener esta información.

Tabla 5.14: Sprint 6 - Uso GS - Planes - Planes por Tipos

Historia de usuario	
Número: 15	**Esfuerzo**: 5
Nombre historia: Uso GS - Planes - Planes por Tipos	
Descripción: Como usuario quiero ver datos sobre la evolución temporal del número de planes que hay de cada tipo	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añaden en la pestaña de *Uso GS* tres gráficas que muestren el número de planes de continuidad, gestión de crisis y recuperación por cada empresa. Además, estas gráficas deben ser filtrables por fecha y por esquema.	



Figura 5.10: Ejemplo de descripción emergente

Durante la tarea 5.16 se presentó un nuevo problema. Se querían presentar los datos del último día con registros disponible pero, al haber datos disponibles de días anteriores, surgía un nivel de complicación mayor al de solamente mostrar datos en la gráfica. En un inicio se intentó hacerlo con una opción llamada *filter* que proporciona MongoCharts, que permite coger los datos de una fecha declarada de forma relativa, es decir, permite coger datos del último día con respecto a hoy. Esta solución nos daba únicamente la posibilidad de coger los datos de "ayer", pero de no haber recogido datos en ese lapso de tiempo, la gráfica quedaría vacía, lo cual no nos interesaba. Es por ello que se utilizó otra opción de MongoCharts, una que permite ejecutar agregaciones sobre los datos de una colección antes de crear la gráfica (Ver Listado 5.8).

También durante la tarea 5.16 hubo que modificar los filtros de estas gráficas ya que únicamente deberían responder a los cambios de esquema, no a los cambios de fecha.

Tabla 5.15: Sprint 6 - Uso GS - Planes - Activación de la Crisis

Historia de usuario	
Número: 16	**Esfuerzo**: 5
Nombre historia: Uso GS - Planes - Activación de la Crisis	
Descripción: Como usuario quiero ver datos sobre la evolución temporal del número de crisis activadas, crisis que desencadenan una fase de recuperación y simulacros de crisis realizados	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añaden en la pestaña de *Uso GS* tres gráficas que muestren, el número de crisis activas, crisis que desencadenan una fase de recuperación y simulacros realizados. La primera gráfica únicamente de las empresas con licencia *Continuity* o *PIC*. Además, estas gráficas deben ser filtrables por fecha y por esquema.	

Tabla 5.16: Sprint 6 - Uso GS - Empresas - Datos actuales de empresas

Historia de usuario	
Número: 17	**Esfuerzo**: 2
Nombre historia: Uso GS - Empresas - Datos actuales de empresas	
Descripción: Como usuario quiero ver datos sobre el estado actual del número de empresas que hay de alta, baja, formación de alta y formación de baja, así como de los tipos de empresa que hay de alta y baja.	
Definición de Hecho: La historia de usuario se podrá dar por completada si se añaden en la pestaña de *Uso GS* tres gráficas que muestren, una que muestre las empresas de alta y baja que hay hoy, otra que muestre las empresas que hay de formación en alta y baja, y por último, los tipos de empresa que hay de alta y baja. Además, estas gráficas deben ser filtrables por esquema.	

Listado 5.8: Query para sacar los datos de la última fecha

```
[
    {
        $group: {
            _id: '$timestamp',
            lastDate: {
                $max: '$timestamp'
            },
            metadata: {
                $addToSet: '$metadata'
            }
        }
    },
    {
        $sort: {
            lastDate: -1
        }
    },
    {
        $limit: 1
    },
    {
        $unwind: {
            path: '$metadata'
        }
    }
]
```

Esta agregación tiene varias etapas:

1. Agrupar los datos por su campo $timestamp, esto genera un documento por cada timestamp distinto, y crea un campo llamado *lastDate* que contiene el $timestamp más cercano a la fecha actual y un campo *metadata*, que gracias al operador *$addToSet*

incluye en el campo *metadata* todos los campos *metadata* de los documentos que ha agrupado.

2. Ordena los documentos por fecha (*$lastDate*) en orden descendente. De más nuevo a más viejo.
3. Usa *$limit : 1* para coger el primer documento de la lista que, al haber sido ordenada antes por fecha, contiene el documento con los datos de la fecha más cercana al día actual.
4. Utiliza el operador *$unwind* para desenrollar lo contenido en la *array metadata*. Esto nos deja por un documento por cada posición en la *array* cuya fecha es la que queremos.

Gracias a lo explicado en el Listado 5.7 podemos mostrar los datos del último día contenido en la base de datos.

Tabla 5.17: Sprint 6 - Modificar tabla Surveys

Historia de usuario	
Número: 18	**Esfuerzo**: 1
Nombre historia: Modificar tabla Surveys	
Descripción: Como desarrollador quiero eliminar el *bug* que añade una fecha erronea en el campo *launched* y *expired*, así como modificar la query para que en vez de contar, sume las ocurrencias de 1 y 0 para calcular los campos *finished* y *consolidated*	
Definición de Hecho: La historia de usuario se podrá dar por completada si La tarea será completada cuando los campos *launched* y *expired* contengan la cantidad de encuestas terminadas y consolidadas por empresa, y los campos *launched* y *expired* las fechas correctas.	

Tabla 5.18: Sprint 6 - Eliminar el *endpoint* de ayuda al desarrollo

Historia de usuario	
Número: 19	**Esfuerzo**: 1
Nombre historia: Eliminar el *endpoint* de ayuda al desarrollo	
Descripción: Como desarrollador quiero eliminar los *endpoints* de ayuda al desarrollo para generar una versión en la que no se pueda invocar a los métodos de debug.	
Definición de Hecho: La historia de usuario se podrá dar por completada si Se borra el controlador y la carpeta de debug	

La tarea 5.18 constó de la eliminación del código que se había estado utilizando para realizar llamadas a métodos de la aplicación cuando se querían debuggear. Al ser los métodos, en un principio, invocados por el *Scheduler*, estos solo se podían ejecutar de dos maneras: Ejecutando el comando *docker exec -it nombre_contenedor php artisan nombre_comando* , o desplegando en un entorno en el que el *Scheduler* estuviese configurado para ver si existían nuevos datos después de la hora a la que se hubiera programado. Esto suponía un problema a la hora de debuggear código puesto que el debugger no estaba configurado para ninguna de las dos casuísticas. Debido a esto, vimos oportuno crear un *endpoint* para invocar a los

métodos mediante el protocolo *HTTP* y poder activar el debugger realizando llamadas desde *Postman*. Una vez se finalizase el desarrollo, se eliminaría este *endpoint* y se generaría una versión sin él.

La relación entre las tareas mencionadas arriba y las capturas de pantalla que muestran los resultados de cada una de ellas, con los datos anonimizados de acuerdo al documento de confidencialidad firmado con la empresa, se muestra en la tablas 5.19

Tarea	Captura
5.10	5.11
5.11	5.12
5.12	5.13
5.13	5.14
5.14	5.15 y 5.16
5.16	5.18

Tabla 5.19: Relación entre las tareas del Sprint 6 y las capturas de pantalla resultantes de cada una de ellas

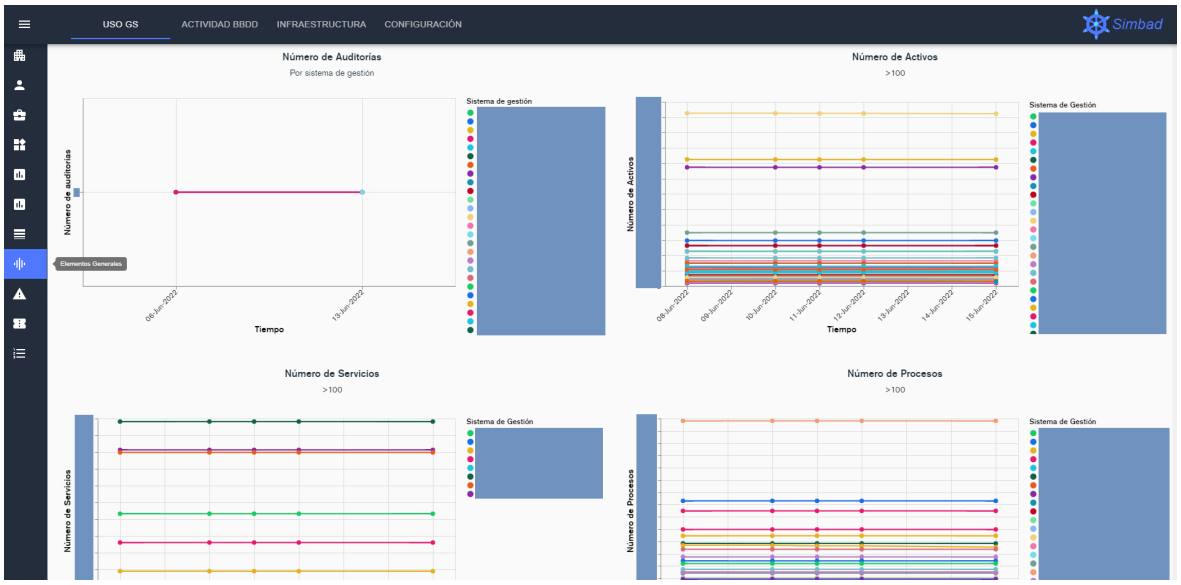


Figura 5.11: Pestaña resultado de la tarea 5.10

Finalmente, era necesario que las gráficas se viesen en producción. Para ello, había que hacer varias cosas: Primero, exportar las gráficas del cuadro de mandos local al cuadro de mandos de producción; segundo, cambiar las colecciones de las que se cogen los datos, quitando las de local y poniendo las de producción; tercero, dar permisos a las gráficas y fuentes de datos para que se pueda ver en la aplicación web; cuarto y último, poblar la colección en la que encontramos los identificadores de las gráficas en la colección de la base de datos de producción.

Tras la realización de este *Sprint* comenzó una etapa de revisión estética de la aplicación para prepararla para su despliegue sin errores.

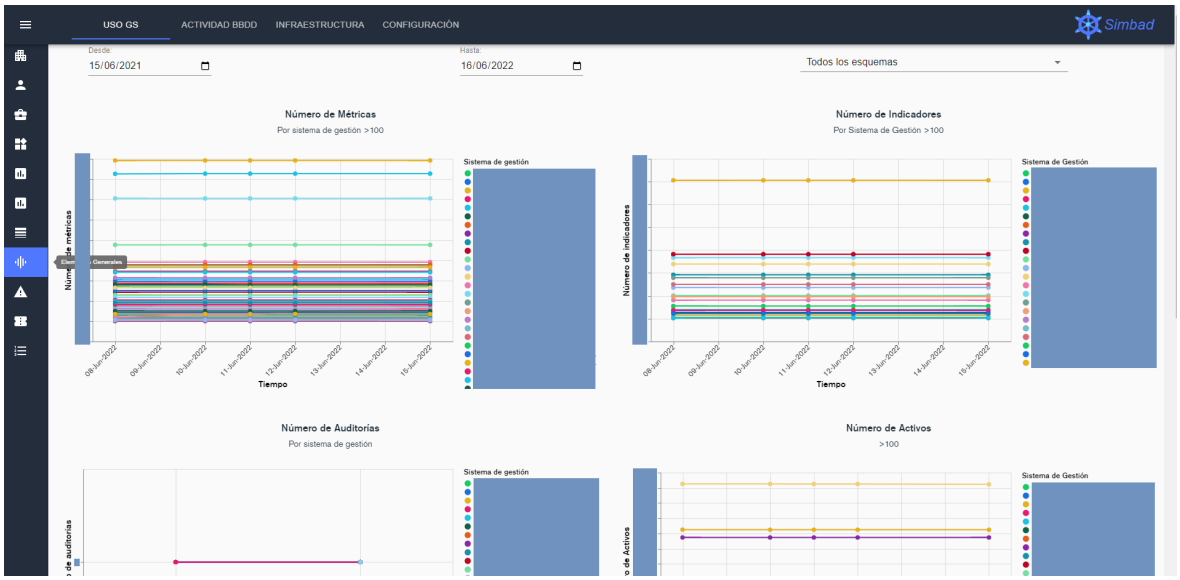


Figura 5.12: Pestaña resultado de la tarea 5.11

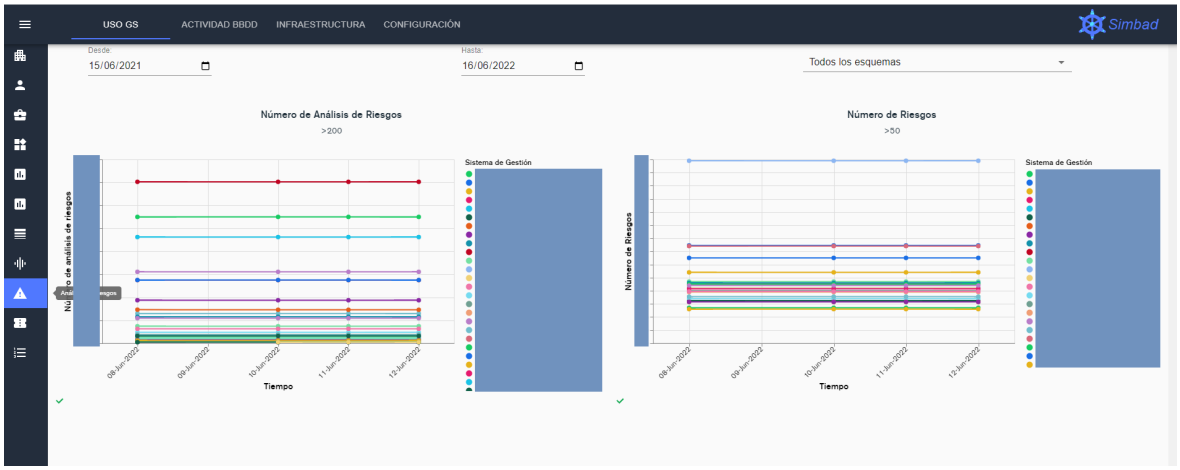


Figura 5.13: Pestaña resultado de la tarea 5.12

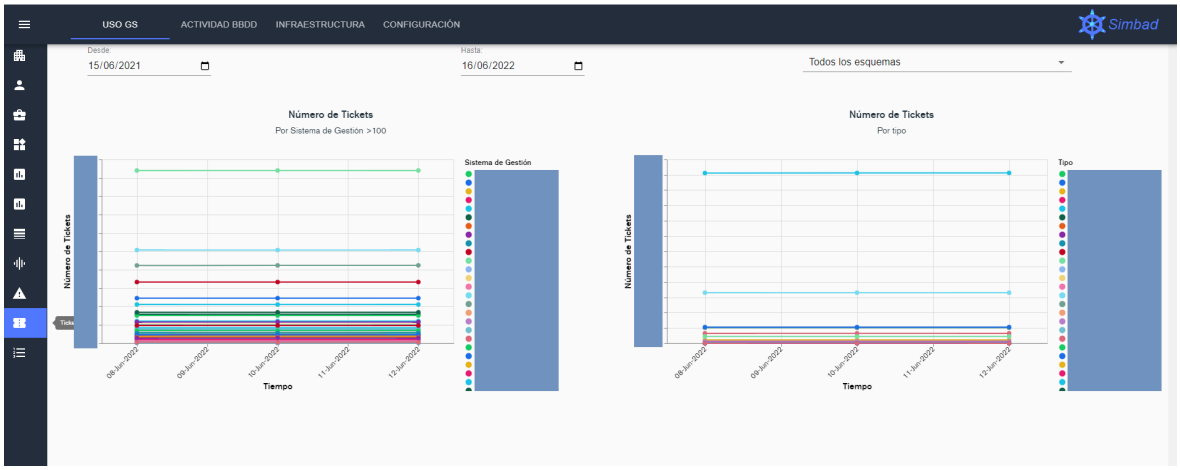


Figura 5.14: Pestaña resultado de la tarea 5.13

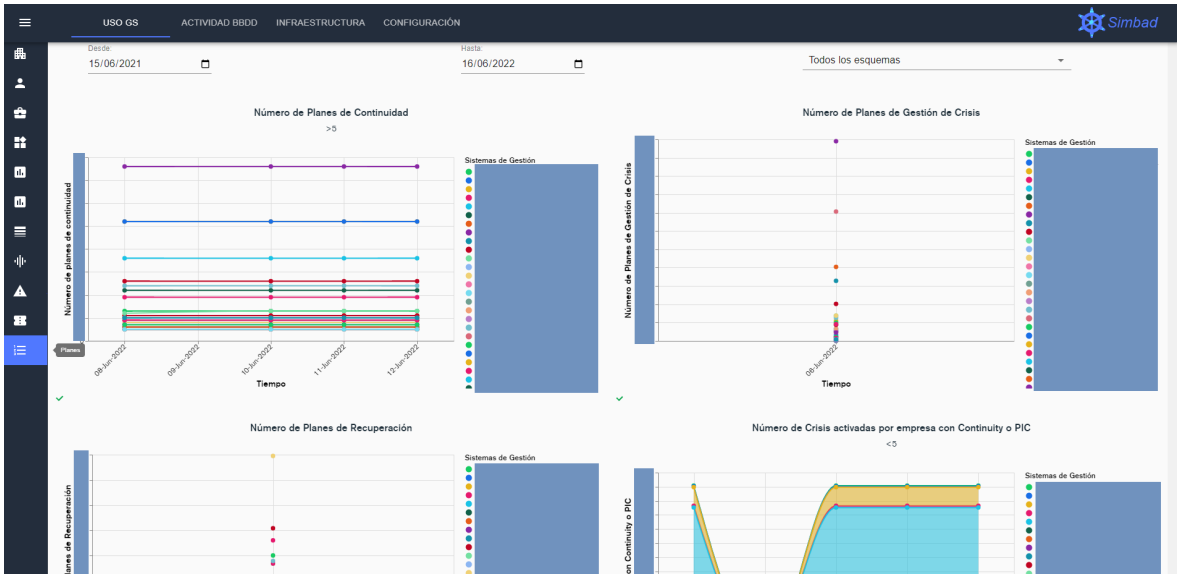


Figura 5.15: Primera parte de la pestaña resultado de la tarea 5.14

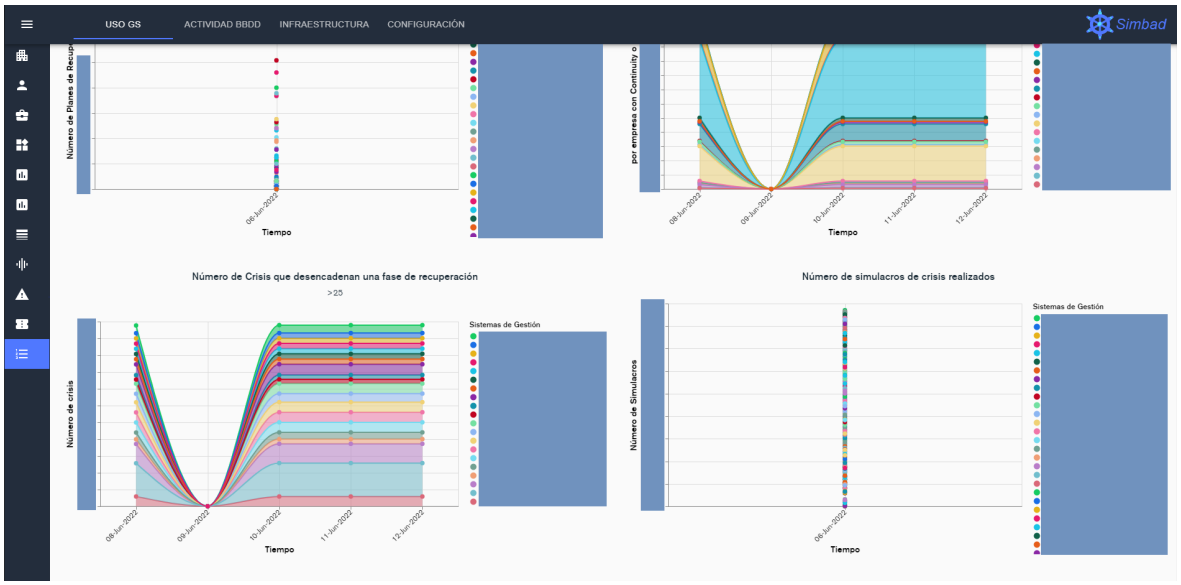


Figura 5.16: Segunda parte de la pestaña resultado de la tarea 5.14

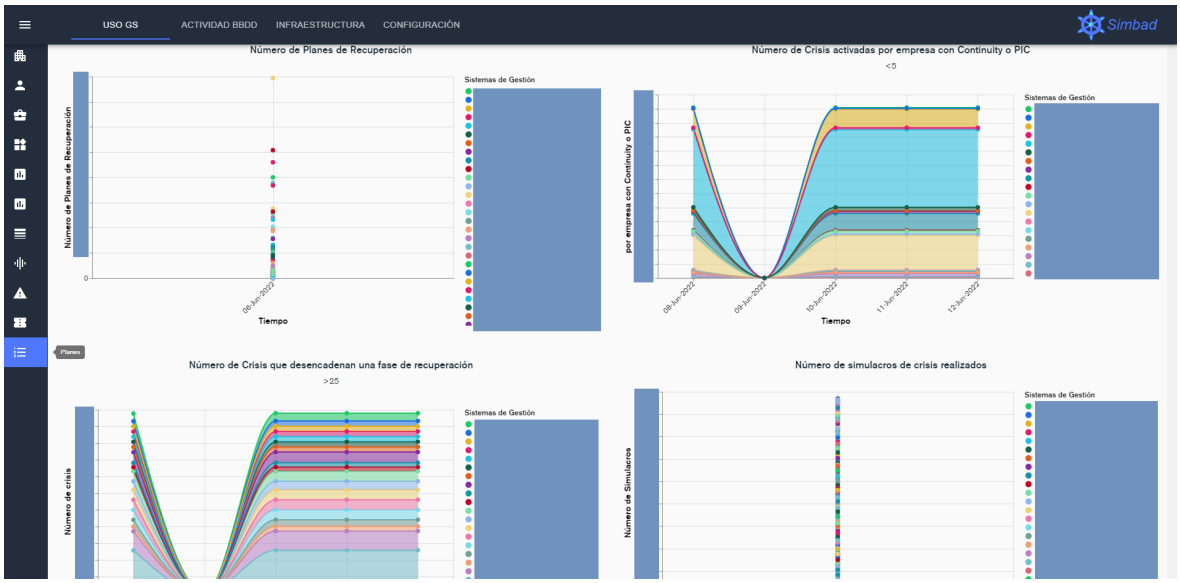


Figura 5.17: Segunda parte de la pestaña resultado de la tarea 5.15

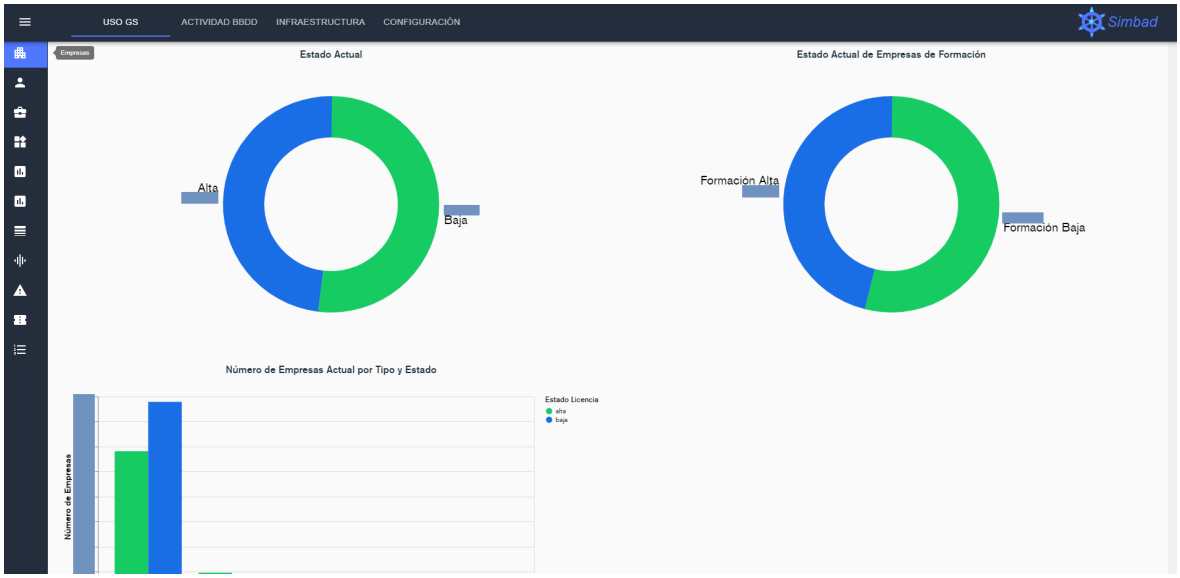


Figura 5.18: Segunda parte de la pestaña resultado de la tarea 5.16

Por último, como ejemplo de seguimiento de la metodología de gestión de proyectos *Scrum*, en la figura 5.19 se puede ver una captura de pantalla de la aplicación usada para hacer las reuniones de retrospectiva[9], donde definimos cuatros columnas:

1. **Went Well:** Se menciona lo que fue bien durante el *Sprint*, se espera que lo aquí mencionado se mantenga en un futuro. Concretamente en esta retrospectiva mencionamos que se han completado bastantes tareas, que se ha acotado mejor el esfuerzo planificado en *Sprint*, que ha mejorado la estructura del cliente de la aplicación Cronos, que conocemos y sabemos usar nuevas tecnologías como *Redux*, y que la comunicación y colaboración entre los integrantes del equipo ha sido buena.
2. **To Improve:** Se mencionan las cosas a pulir o a mejorar, en concreto en esta etapa comenté que me gustaría seguir mejorando la calidad de mi código y las *querys* que creaba, no porque fueran de mala calidad, sino para seguir perfeccionándolo y haciéndolo en menos tiempo.
3. **Problems:** Se mencionan los problemas abordados durante el Sprint, ya sean los resueltos o los pendientes a resolver. En este Sprint hubo problemas con la CI que no dependían de nosotros y esto nos retrasó, pero el problema quedó resuelto, también comenté que hay casuísticas de programación que no controlo, problema comprensible ya que PHP sigue siendo un lenguaje relativamente nuevo para mi, y que quedaron resueltos durante el *Sprint*. Posteriormente se comentó que existen tareas de DBA(Database Administration) difíciles de estimar puesto que la duración de esto puede variar mucho dependiendo de la naturaleza del problema, y esta naturaleza no se conoce hasta que se ha comenzado a abordar el problema. Por último, se mencionó que existen fragmentos de código difíciles de probar, esto es debido a que, aun habiendo hecho pruebas anteriormente, existían funciones o llamadas que no eran fácilmente probables por su dependencia, por ejemplo, de *Laravel*
4. **Wishes:** Esta columna contiene los deseos que tenemos, lo que nos gustaría que ocurriese en un futuro. Las tarjetas que se incluyeron mencionaban el deseo de que MongoDB mejorase los tiempos de carga de las gráficas, ya que es mayor al deseado, seguir avanzando con el desarrollo y la escritura de este TFG, y por último, mi incorporación al equipo para realizar, además de tareas relacionadas con este proyecto, tareas relacionadas con la administración de bases de datos.

Posteriormente a la inclusión de las tareas, se van leyendo las tarjetas que se han añadido y se comenta entre todos.

[9] https://reetro.io/



Figura 5.19: Tablero la reunión retrospectiva correspondiente al *Sprint* 6. Algunos datos están censurados en base a los acuerdos con la empresa.

CAPÍTULO 6

Conclusiones

Durante este capítulo se exponen los objetivos conseguidos durante el desarrollo del proyecto, además, se incluyen las competencias obtenidas y una valoración general del proyecto.

6.1. CONSECUCIÓN DE OBJETIVOS PARCIALES

6.1.1. Creación de la base de datos que contenga los datos históricos.

Durante el *Sprint 0* (ver sección 5.1), el de configuración, se creó un *Cluster* con varias bases de datos, una por cada entorno productivo. Posteriormente, en los sucesivos *Sprints* se fueron creando colecciones para contener los datos.

Al final de proyecto quedan creadas y pobladas un total de quince colecciones. Algunas se han utilizado para una única tarea, otras se reutilizaron, y otras se han ampliado.

6.1.2. Análisis de las fuentes de datos

En distintas tareas de las sub-secciones 5.2, 5.3, 5.4, 5.5, 5.6, 5.7 se han tenido que analizar las fuentes de datos. A la hora de realizar las consultar tanto a MariaDB como a MongoDB, ha habido que analizar donde están los datos, que relación tienen con otras tablas, y qué información nos interesa almacenar de ellas. Utilizando herramientas como HeidiSQL para MariaDB y Robo3T para MongoDB, esta tarea ha resultado más sencilla, aunque dado el gran número de tablas y columnas existentes, alguna vez se ha tenido que preguntar dónde se encontraban ciertos datos a los compañeros.

A modo de ejemplo, el Listado 5.8, que aplica una agregación a los datos almacenados en MongoDB, es resultado del análisis de esta colección para ver qué tiene, y qué queremos obtener.

6.1.3. Implementación de métricas

Durante las tareas contenidas en los *Sprints* desglosados en las sub-secciones 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, se ha implementado el código para implementar las métricas, en estas tareas se obtenían los datos de las fuentes pertinentes, luego estos datos se transformaban para

agruparlos según algún parámetro, como por ejemplo el sistema de gestión al que pertenecen, y finalmente estos datos agrupados se añadían a la colección necesaria.

Esta etapa ha sido la encargada de poblar las colecciones de MongoDB y la que se ejecutaba periódicamente para obtener los datos.

Un ejemplo de agrupación de datos se puede encontrar en el listado 5.2, y parte del código usado para tratar los datos se encuentra en el listado 5.5.

6.1.4. Creación de un cuadro de mando para la presentación de los datos.

Las tareas contenidas en las sub-secciones 5.2, 5.3, 5.4, 5.5, 5.6, 5.7 , han ido orientadas a esto, a crear una plataforma web donde ver métricas y poder aplicarle a estas filtros que modifiquen el esquema de la base de datos o la serie temporal que se muestra.

El resultado final se puede ver, entre otras, en las figuras 5.11, 5.12, 5.13, 5.14, 5.15, 5.16, 5.17 y 5.18.

6.2. CONSECUCIÓN DE OBJETIVOS DOCENTES

6.2.1. Capacidad de trabajo con bases de datos tanto relacionales como no relacionales. Creación de estas, realización de consultas y búsqueda de registros.

Este objetivo se ha completado durante el proyecto. Se han realizado consultas aplicando lo aprendido en las asignaturas con bases de datos tanto de SQL como de NoSQL al no profundizar durante el grado en MQL (*Mongo Query Language*), se ha aprendido bastante sobre la adición, consulta y manejo de datos con este lenguaje.

- **Relacionales.** Se han realizado consultas utilizando SQL, todas ellas de obtención de datos y ninguna de introducción. Para ello, se han tenido que usar clausulas como *WHERE* y *HAVING*, o sentencias como *DISCTINCT*, *SUM*, etcétera. Además, se han tenido que combinar datos de distintas tablas, para lo cual se ha hecho uso de *INNER JOIN* y *LEFT JOIN*
- **No Relacionales.** Usando MQL, se han tanto añadido, consultado y filtrado datos. A la hora de añadir, se ha hecho uso de la función *insertMany()*, para consultar datos se ha utilizado la función *find()* o *aggregation()* dependiendo del uso que se quería explotar, y para filtrar se ha utilizado *aggregation()* (ver figura 5.8) puesto que era la que mejor se adaptaba a nuestras circunstancias.

6.2.2. Seguimiento de metodologías ágiles con su respectiva adaptación al contexto del equipo.

Durante el transcurso del proyecto se han seguido metodologías ágiles, concretamente, se ha adaptado *Scrum* a nuestras necesidades. Tanto la implementación como los roles que asumimos cada uno de los integrantes del equipo quedan explicados en la sección de

metodología dedicada a esta metodología de gestión de proyectos (ver sección 4.1). Además, se ha incluido el resultado de la reunión de retrospectiva del *Sprint* 6 (ver sección 5.7) en la imagen 5.19

6.2.3. Seguimiento de las buenas prácticas a la hora de programar, manteniendo un código limpio, legible, comprensible, documentado y de calidad

Para la consecución de este objetivo, se ha implementado varios métodos. Lo primero, a la hora de configurar el entorno de programación, se configuran también algunas normas que debe cumplir el código para no mostrar un aviso. Esto contiene tanto restricciones sobre la identación de los bloques de código, como el uso de *camelCase* por poner algún ejemplo. Después, otro control que pasa el código son las revisiones de código que hacen los compañeros, donde si hay algo que no cumpla los requisitos de calidad, se indicará y se tendrá que cambiar. En esta etapa se cuenta con la ayuda de *sonarQube*, que esta configurado para que a la hora de intentar integrar nuevo código en una rama, este no pueda tener *code smells* (ver figura 5.9).

6.2.4. Aplicación de los conocimientos adquiridos sobre la creación, necesidad y utilidad de los cuadros de mando

Como se puede ver en figuras como 5.6 o 5.8, el proyecto ha dado pie a la creación de un cuadro de mando donde se muestran gráficas que contienen datos, así como selectores de fecha y esquema que cambian los datos de las gráficas aplicando filtros.

6.2.5. Desarrollo de sistemas que puedan procesar y responder peticiones HTTP

El proyecto, en la parte de la API, tiene código capaz de procesar y responder peticiones HTTP gracias a que PHP bajo el marco de trabajo de *Laravel*. Existe una clase que enruta las peticiones a los controladores, y estos gestionan las peticiones según lo que necesita. El uso principal que se le da durante el proyecto es el de devolver los identificadores de las gráficas a mostrar, como se indica en el *Sprint 5* (ver sección 5.6).

6.2.6. Uso de software de control de versiones

El uso de *sofware* de control de versiones se ha utilizado durante todas las tareas. Se ha hecho uso de *Bitbucket* para contener los repositorios de código, como es lógico, pero también para controlar la calidad de este y dar soporte a la integración continua (*Continous Integration*). La calidad gracias a su integración con SonarQube (ver figura 5.9) y gracias a que da soporte a que los desarrolladores realicen revisiones de código sobre las *Pull Request*, y el soporte a la CI gracias a que está integrado con Jenkins.

6.3. CONSECUCIÓN DEL OBJETIVO GENERAL

Como ha quedado demostrado en las subsecciones 6.1 y 6.2, el objetivo de **crear un sistema de almacenamiento, procesamiento y presentación de datos históricos relacionados con el *software* GlobalSuite, que permita identificar e implementar mejoras técnicas y de negocio analizando tendencias sobre el uso y gestión del producto** se ha conseguido completar.

Como se ha ido mostrando, la aplicación ha ido incrementando su valor conforme avanzaba el proyecto. Se han implementado gráficas de varios tipos, con datos de varias fuentes y adquiridos de forma automática, a las que se le pueden aplicar filtros y de los que se pueden consultar los valores.

6.4. ADQUISICIÓN DE COMPETENCIAS ESPECÍFICAS

Durante el transcurso de las prácticas se han adquirido numerosas competencias y habilidades relacionales con lo adquirido durante la etapa universitaria.

Primero de todo, se ha reforzado la habilidad del trabajo en equipo, ya que he formado parte de un equipo y, aunque el desarrollo haya sido propio, he asistido a reuniones, preguntado dudas y mi código ha sido revisado por la gente del departamento. También he complementado mis conocimientos sobre bases de datos, tanto relacionales como no relacionales, y tanto de gestión como de uso. Otra cosa que se me ha enseñado durante la carrera y que se ha visto utilizada durante esta etapa de prácticas es la de tener capacidad de adaptación y aprendizaje a nuevas tecnologías. Siguiendo con conocimientos adquiridos en la carrera y aplicados en prácticas, la creación de aplicaciones web que usen el protocolo HTTP. Por último, en cuanto a habilidades, se ha trabajado aplicando conocimientos en código limpio y calidad de software con herramientas como SonarQube.

Hablando de competencias específicas de la Tecnología Específica de Ingeniería del Software cabe reseñar la consecución de algunas de ellas:

[IS1] Capacidad para desarrollar, mantener y evaluar servicios y sistemas software que satisfagan todos los requisitos del usuario y se comporten de forma fiable y eficiente, sean asequibles de desarrollar y mantener y cumplan normas de calidad, aplicando las teorías, principios, métodos y prácticas de la Ingeniería del Software: Esta competencia se ha adquirido puesto que el producto final resultante del trabajo en el proyecto ha sido fructífero. Se han completado las tareas planificadas aplicando las teorías, principios, métodos y prácticas de la Ingeniería del Software, y todo bajo un marco ágil de gestión de proyectos como es Scrum. Durante el transcurso del proyecto se ha controlado la calidad del software utilizando herramientas como sonarQube, se ha trabajado en un entorno con CI y se ha utilizado un software de control de versiones como es BitBucket, por mencionar algunos ejemplos.

[IS3] Capacidad de dar solución a problemas de integración en función de las estrategias, estándares y tecnologías disponibles: La adquisición de esta competencia se justifica en que los problemas abordados durante la ejecución del proyecto se han solventado utilizando las estrategias, estándares y tecnologías disponibles. Se han utilizado desde tecnologías como pueden ser los lenguajes PHP o JavaScript con sus correspondientes características, hasta estrategias como el bucketing para optimizar el almacenamiento. Sin olvidar las dos distintas tecnologías de bases de datos utilizadas, con MariaDB sistema de gestión relacional y MongoDB como no relacional.

[IS4] Capacidad de identificar y analizar problemas y diseñar, desarrollar, implementar, verificar y documentar soluciones software sobre la base de un conocimiento adecuado de las teorías, modelos y técnicas actuales: Esta competencia se ha justificado en la utilización de herramientas que, hoy en día, están vigentes en el desarrollo de *software*. Desde PHP que, pese a su longevidad, sigue siendo muy utilizado hoy en día, hasta la aplicación de tecnologías muy novedosas como son las colecciones de tipo *Time Series* de MongoDB. Es por esto por lo que se puede decir que el proyecto esta actualizado y a la orden del día en cuanto a tecnologías. Si a eso le sumamos que el proyecto en si trata de dar soporte a la analítica de datos, podemos decir que también está actualizado en cuanto a concepto.

6.5. EL FUTURO DE CRONOS

Tras la finalización del proyecto que se describe durante este TFG, la empresa se ha ofrecido a continuar con nuestra relación laboral con la vista puesta en que siga formando parte del departamento de bases de datos. Dentro de este departamento se ha marcado el objetivo de continuar avanzando en el campo de la analítica de datos para lo que se ha establecido como hito la implementación de un lago de datos utilizando tecnologías en la nube de la mano de AWS (Amazon Web Services).

AWS es una *suite* de Amazon que incluye servicios entre los que destacamos algunos como S3 (Almacenamiento), Glue (Clasificación) y Athena (Consulta). Estos servicios, junto con otros más específicos, aportan las herramientas necesarias para construir un lago de datos en el que poder almacenar y clasificar datos para luego procesarlos y mostrar información.

El futuro de cronos va ligado a esto. Dentro de una lago de datos pueden guardarse almacenes de datos, y como cronos actúa como un almacén de datos, este se puede añadir como fuente. Además, todo el conocimiento adquirido durante la realización del proyecto se puede aplicar a la creación de este lago de datos, tanto el conocimiento teórico en materia de análisis de datos, como el conocimiento práctico adquirido sobre gestión de bases de datos (ver apartado 6.2.1), o de implementación (ver apartado 6.2.4). Esto sumado a la buena integración en el equipo de trabajo hace que la linea por la que se quiere continuar sea clara, actualizada, y útil para el negocio.

6.6. VALORACIÓN CRÍTICA

La valoración del proyecto es positiva. He conseguido superar los problemas que se han presentado para llegar a un producto de calidad sobre el que se puede seguir trabajando. Tras la finalización de este proyecto me queda claro que puedo enfrentarme a retos nuevos ya que el conocimiento adquirido durante mi etapa universitaria me aporta el conocimiento necesario para esto.

Sin duda, una de las cosas que más me ha aportado este proyecto es la realización de este dentro de un entorno real, una empresa con los procesos de desarrollo bien definidos y en la que los compañeros tienen una actitud receptiva. Esto hace que se asienten los conocimientos adquiridos durante el grado, se aprendan cosas nuevas de la mano de gente con experiencia y se viva desde dentro un flujo de trabajo que concluye con el producto esperado desplegado.

Es por esto que recomiendo la experiencia de realizar un FORTE a los demás alumnos del grado, lo veo una experiencia positiva y que complementa muy bien los estudios.

Bibliografía

[1] Surajit Chaudhuri and Umeshwar Dayal. An overview of data warehousing and olap technology. *SIGMOD Rec.*, 26(1):65–74, mar 1997.

[2] E. F. Codd. A relational model of data for large shared data banks. *Commun. ACM*, 13(6):377–387, jun 1970.

[3] Natalia Miloslavskaya and Alexander Tolstoy. Big data, fast data and data lake concepts. *Procedia Computer Science*, 88:300–305, 2016. 7th Annual International Conference on Biologically Inspired Cognitive Architectures, BICA 2016, held July 16 to July 19, 2016 in New York City, NY, USA.

[4] Christof Strauch, Ultra-Large Scale Sites, and Walter Kriha. Nosql databases. *Lecture Notes, Stuttgart Media University*, 20(24):3–8, 2011.